Andrea Solís Moreno 1ºDAW
T4. Actividad 2. Base de Datos.
26 de enero 2026

Actividad 2.

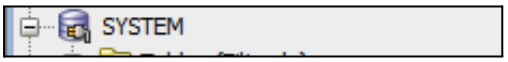
Nota→ Observa el orden en el que creamos las tablas. Las tablas que contienen claves foráneas deben crearse después de crear las tablas que contienen las claves primarias a las que apuntan.

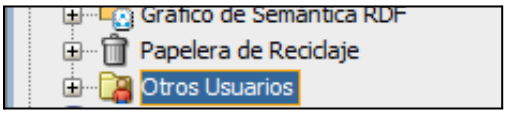
Para cada ejercicio debes crear:

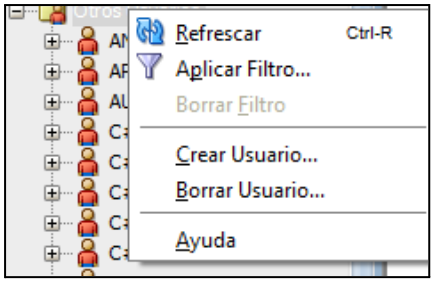
● un usuario que comience por tus iniciales seguidas del número de la actividad.
por ejemplo en caso de Andrea Solís Moreno sería C##ASMA1

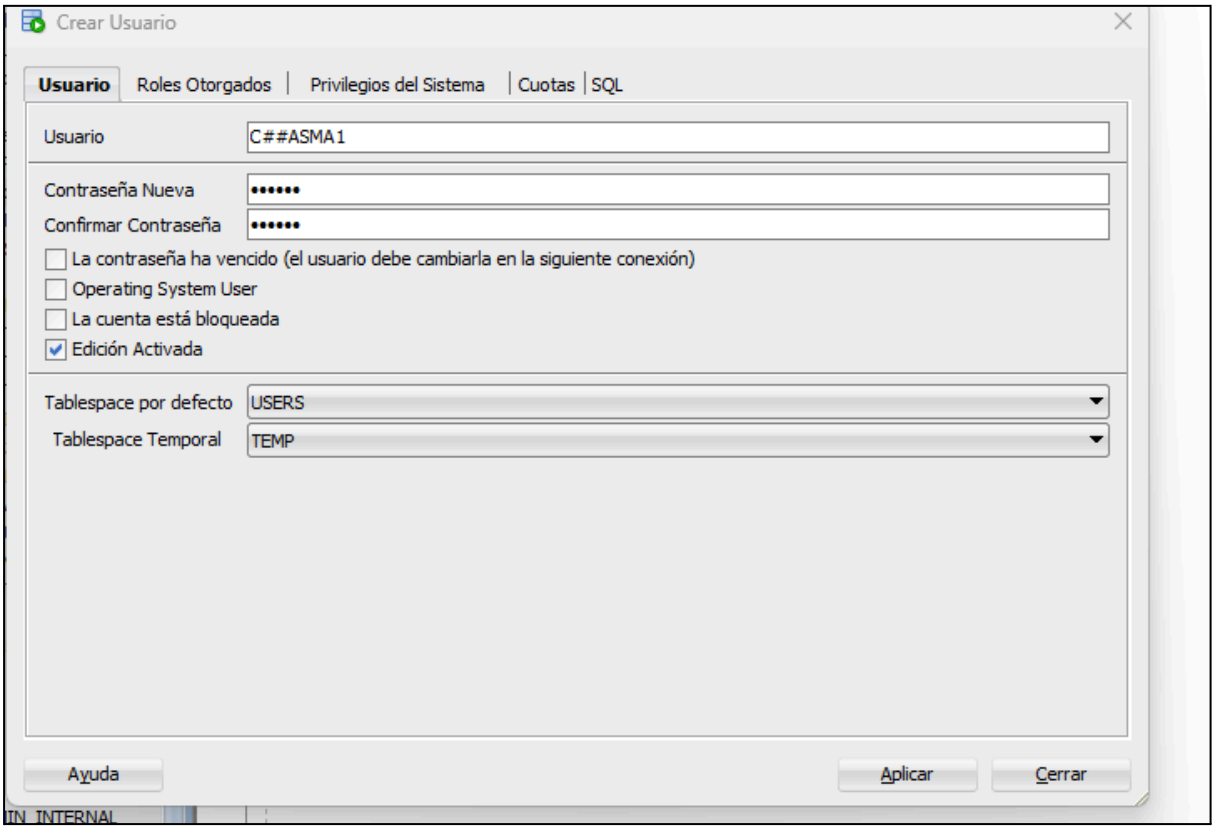
● crear un archivo script .sql que contenga toda la creación de la bases de datos.

SYSTEM → Abajo del todo → Otros usuarios → Botón derecho → Crear usuario

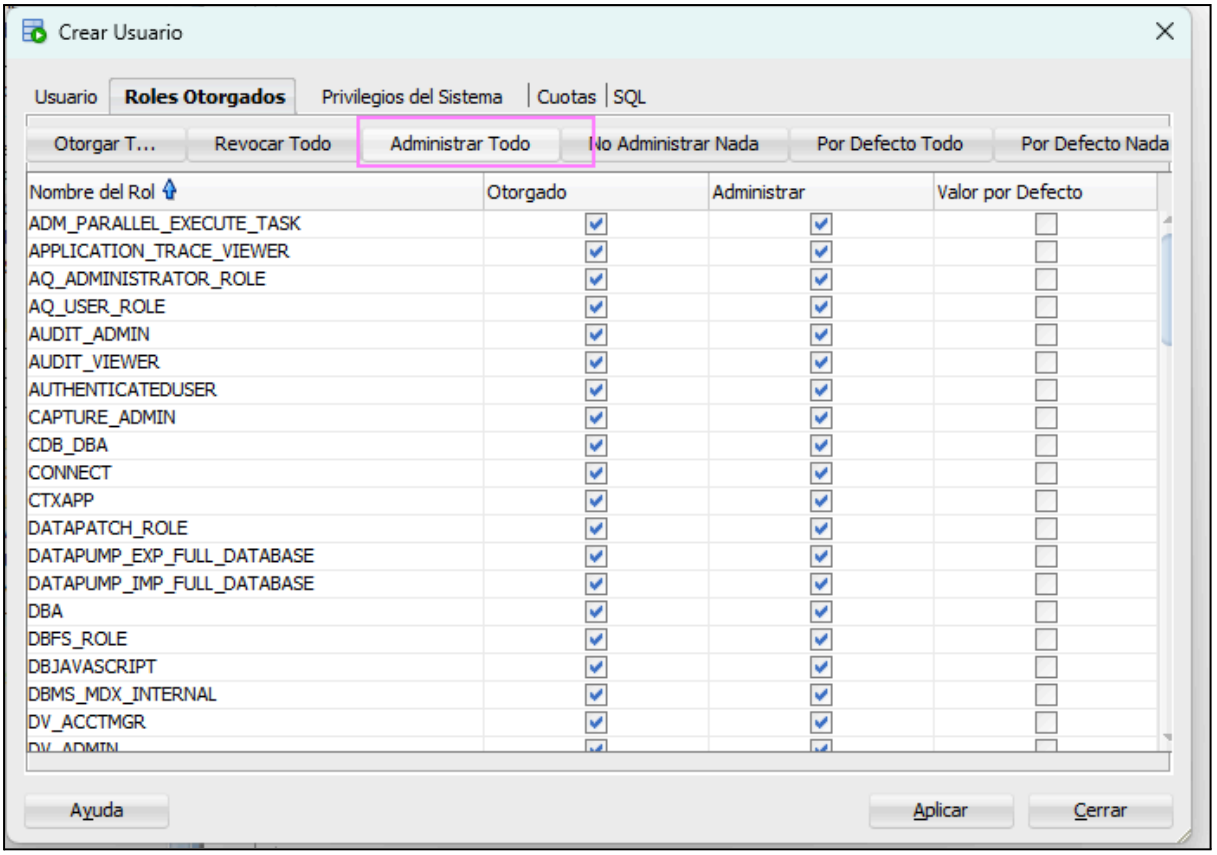






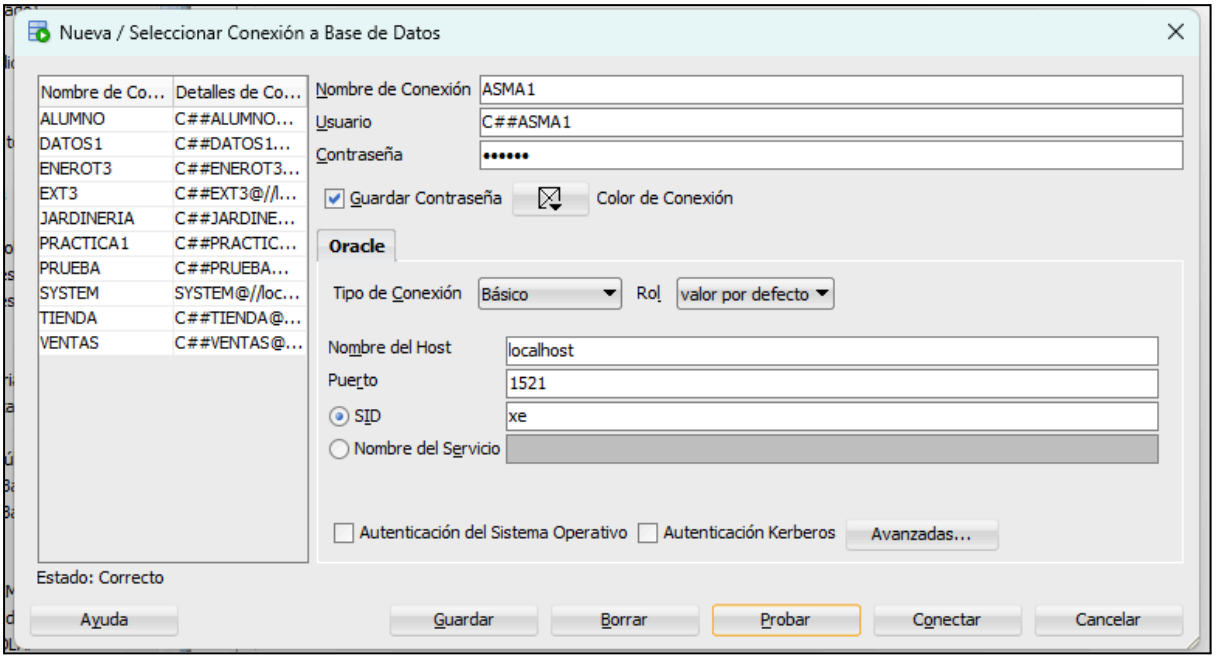


ANTES DE DARLE A APLICAR



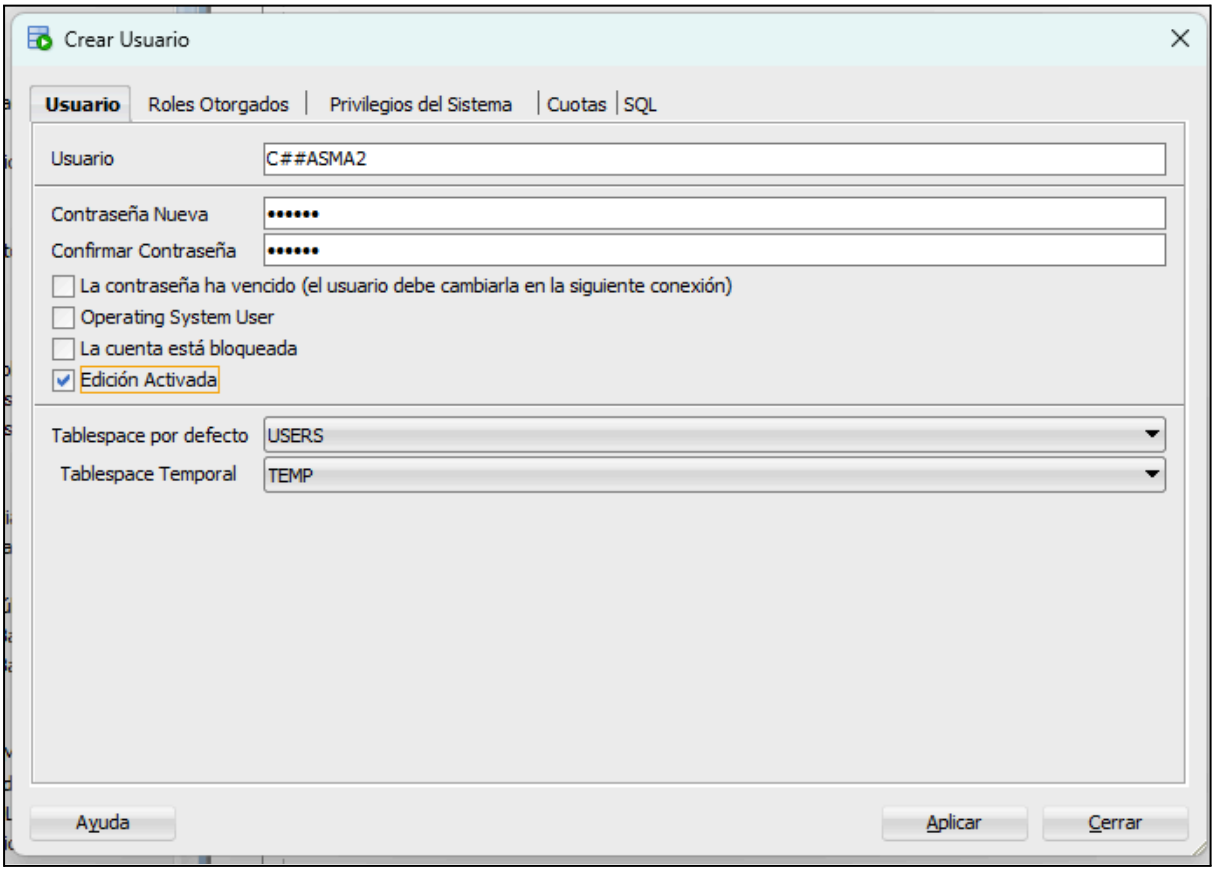
→Refrescamos y nueva conexión

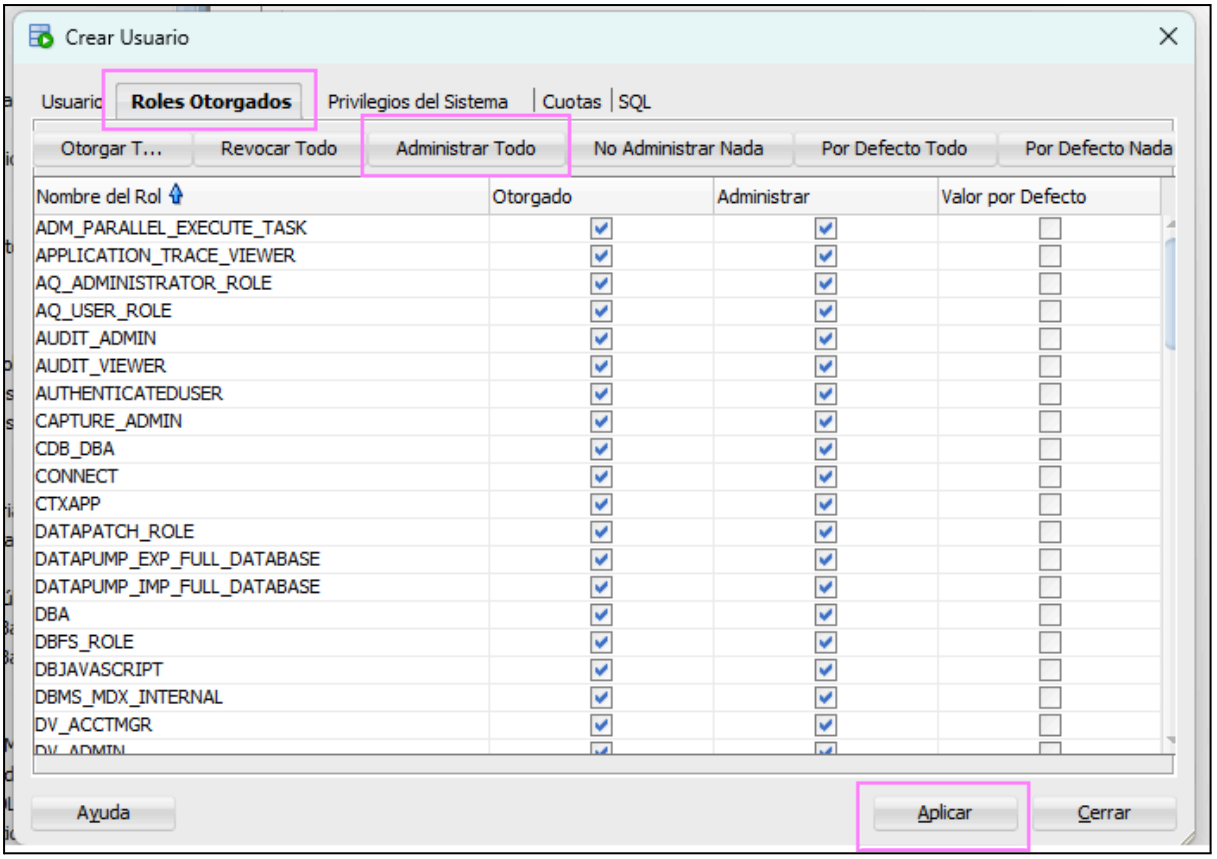




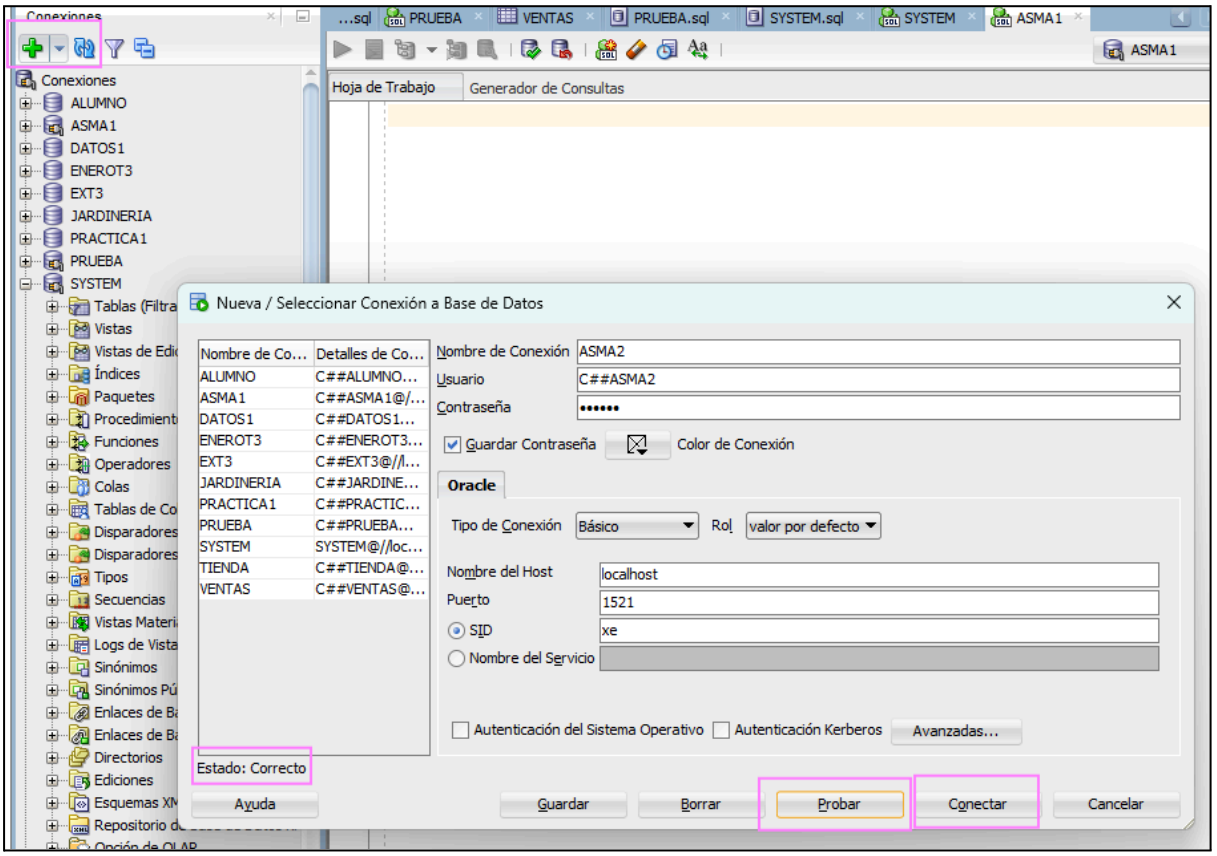
Volvemos a hacer lo mismo para la actividad 2.

Recordemos → System → Otros usuario → Botón derecho→ Crear Usuario

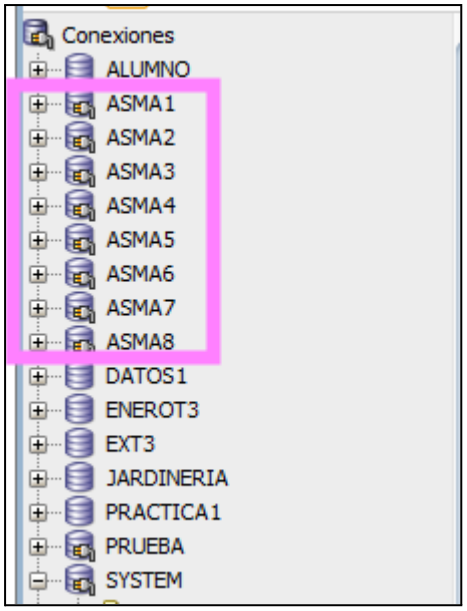




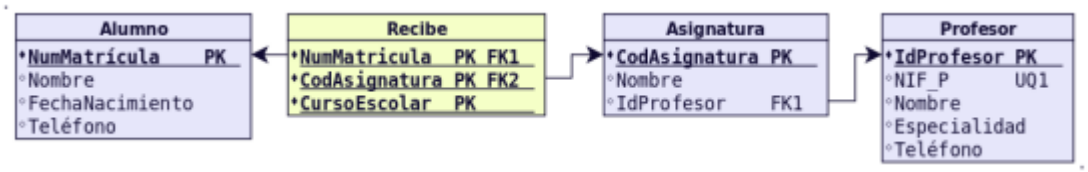
→Refrescar → Nueva conexión



Así con todos los ejercicios.



1. Realiza el diseño físico para el siguiente modelo relacional. Asigna el tipo de datos que consideres más adecuado. Esquema C##TUSINICIALESA1, contraseña "123456"



1. Creación del Usuario (Esquema)

```
-- Creamos el usuario con tus iniciales (ejemplo: ASMA1)
CREATE USER C##ASMA1 IDENTIFIED BY "123456";

-- Le damos permisos para que pueda conectarse y crear cosas
GRANT CONNECT, RESOURCE TO C##ASMA1;
-- Cuota ilimitada en el tablespace para que no de error al insertar datos
ALTER USER C##ASMA1 QUOTA UNLIMITED ON USERS;
```

CREATE USER: Comando DDL para dar de alta a un nuevo usuario.

IDENTIFIED BY: Palabra clave para establecer la contraseña.

Nota: Después de esto, un administrador debería darte permisos (como GRANT CONNECT, RESOURCE TO C##MISINICIALESA1) para poder entrar y crear objetos.

Ejecutamos y nos debe salir usuario alterado y gran correcto.

```
Grant correcto.


User C##ASMA1 alterado.
```

Empezamos a trabajar:

2. Creación de Tablas (Orden Lógico)

¡¡¡¡¡¡¡¡¡¡¡¡¡¡¡¡**Ojo**: El orden es vital.

Debemos crear primero las tablas que no dependen de nadie (las "padre") y al final las que tienen claves ajenas (FK). El orden será: 1. Alumno, 2. Profesor, 3. Asignatura (depende de Profesor), 4. Recibe (depende de Alumno y Asignatura).

★ **Tabla Alumno**

Es una tabla maestra, no tiene flechas que salgan de ella hacia otras.

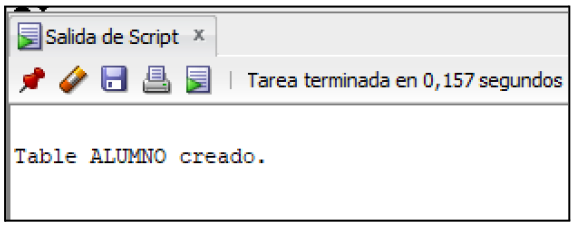
```
CREATE TABLE ALUMNO (
    NumMatricula NUMBER(10),
    Nombre VARCHAR2(50) NOT NULL,
    FechaNacimiento DATE,
    Telefono NUMBER(9),
    CONSTRAINT alumno_pk PRIMARY KEY (NumMatricula)
);
```

Hemos dicho que esta es una tabla básica porque no depende de ninguna otra. Se leería así:

- CREATE TABLE ALUMNO: "Crea una lista para registrar a todos los alumnos".
- NumMatricula NUMBER(10): "Tendremos una columna para el número de matrícula, que admite hasta 10 dígitos".

- Nombre VARCHAR2(50) NOT NULL: "Guardaremos el nombre (hasta 50 letras) y es obligatorio rellenarlo; no puede quedar vacío (NOT NULL)".
- FechaNacimiento DATE: "Registraremos cuándo nació usando el formato de fecha de Oracle".
- Telefono NUMBER(9): "Un campo numérico de 9 dígitos para el teléfono".
- CONSTRAINT alumno_pk PRIMARY KEY (NumMatricula): "La matrícula será el DNI de esta tabla: identifica de forma única a cada alumno y no puede repetirse".

Ejecutamos:




```
describe alumno;
```

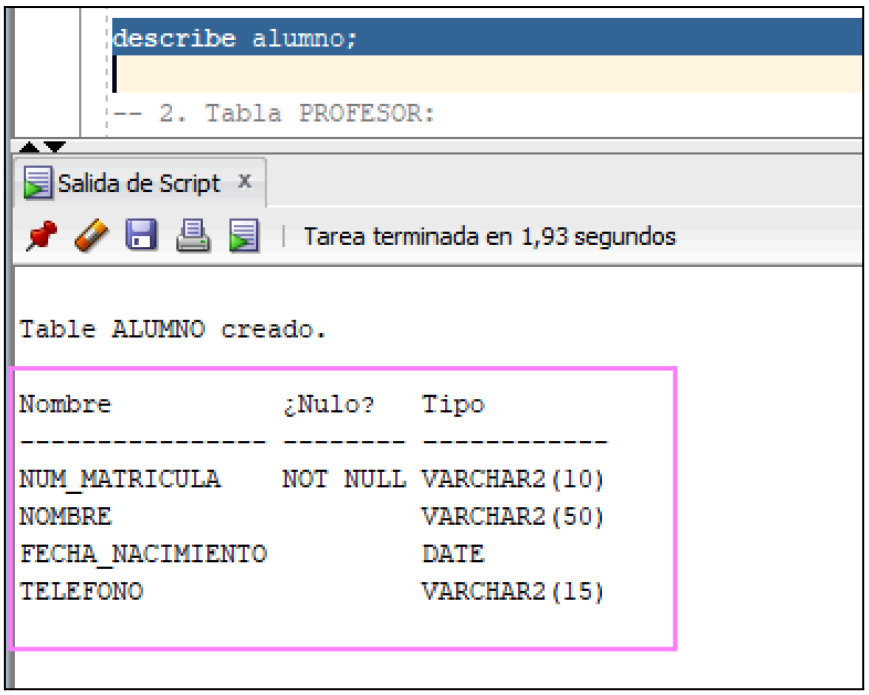
El comando **DESCRIBE** (o su abreviatura **DESC**) sirve para **ver la estructura de una tabla** que ya ha sido creada. Es como pedirle a la base de datos que te enseñe el "plano" o la "receta" de esa tabla.

¿Qué información te da?

Cuando escribes DESCRIBE ALUMNO;, Oracle te mostrará una tabla con tres columnas principales:

1. **Name (Nombre):** El nombre de cada columna que creaste (NUM_MATRICULA, NOMBRE, etc.).
2. **Null? (¿Nulo?):** Te indica si la columna es obligatoria. Si aparece NOT NULL, significa que no puedes dejarla vacía.
3. **Type (Tipo):** Te confirma el tipo de dato y la longitud (por ejemplo, VARCHAR2(10) o DATE).

Si ejecutamos el código de antes y luego pones el comando, verías algo así:



★ **Tabla Profesor**

Otra tabla maestra independiente.

```
--2.TABLA PROFESOR
CREATE TABLE PROFESOR (
    IdProfesor NUMBER(5),
    NIF_P VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Especialidad VARCHAR2(30),
    Telefono NUMBER(9),
    CONSTRAINT profesor_pk PRIMARY KEY (IdProfesor),
    CONSTRAINT profesor_nif_uq UNIQUE (NIF_P)
);
```

- UNIQUE: Impide que dos profesores tengan el mismo NIF, aunque no sea la clave primaria.

Hemos dicho que es otra tabla independiente que sirve de base para las demás.

- CREATE TABLE PROFESOR: "Crea una lista para los profesores".
- IdProfesor NUMBER(5): "Un código numérico de 5 dígitos para identificar al profesor".
- NIF_P VARCHAR2(9): "Su NIF (8 números y una letra)".
- CONSTRAINT profesor_pk PRIMARY KEY (IdProfesor): "El IdProfesor es la clave principal de esta tabla".
- CONSTRAINT profesor_nif_uq UNIQUE (NIF_P): "Aunque el NIF no es la clave principal, prohibimos por ley que haya dos profesores con el mismo NIF (UNIQUE)".

Ejecutamos

```
Table PROFESOR creado.

Nombre       ¿Nulo?   Tipo
------------ -------- ------------
ID_PROFESOR  NOT NULL VARCHAR2(4)
NIF_P                 VARCHAR2(9)
NOMBRE                VARCHAR2(50)
ESPECIALIDAD          VARCHAR2(30)
TELEFONO              VARCHAR2(15)
```

★ **Tabla ASIGNATURA** → DEPENDE DE PROFESOR (RELACIÓN 1:N)

```
--3. TABLA ASIGNATURA
CREATE TABLE ASIGNATURA (
    CodAsignatura NUMBER(5),
    Nombre VARCHAR2(40) NOT NULL,
    IdProfesor NUMBER(5),
    CONSTRAINT asignatura_pk PRIMARY KEY (CodAsignatura),
    CONSTRAINT asignatura_prof_fk FOREIGN KEY (IdProfesor)
        REFERENCES PROFESOR (IdProfesor)
);
```

- FOREIGN KEY (...) REFERENCES: Define la integridad referencial. Indica que el IdProfesor de esta tabla debe existir previamente en la tabla PROFESOR.

Ejecutamos

```
Table ASIGNATURA creado.

Nombre          ¿Nulo?   Tipo
--------------- -------- ------------
COD_ASIGNATURA  NOT NULL VARCHAR2(6)
NOMBRE          NOT NULL VARCHAR2(50)
ID_PROFESOR              VARCHAR2(4)
```

Esta tabla ya empieza a mirar a otras (tiene una "flecha" hacia Profesor).

- CREATE TABLE ASIGNATURA: "Crea una lista de las materias que se imparten".
- CodAsignatura NUMBER(5): "Un código de 5 números para la asignatura".
- IdProfesor NUMBER(5): "Aquí anotaremos qué profesor imparte la asignatura".
- CONSTRAINT asignatura_pk PRIMARY KEY (CodAsignatura): "El código de la asignatura es su identificador único".
- CONSTRAINT asig_prof_fk FOREIGN KEY (IdProfesor) REFERENCES PROFESOR (IdProfesor): "¡Regla importante! El número de profesor que pongas aquí tiene que existir en la tabla de Profesores. No podemos asignar una asignatura a un profesor fantasma".

★ **TABLA RECIBE**

```
--4.TABLA RECIBE
CREATE TABLE RECIBE (
    NumMatricula NUMBER(10),
    CodAsignatura NUMBER(5),
    CursoEscolar VARCHAR2(9),
    CONSTRAINT recibe_pk PRIMARY KEY (NumMatricula, CodAsignatura, CursoEscolar),
    CONSTRAINT recibe_alum_fk FOREIGN KEY (NumMatricula)
        REFERENCES ALUMNO (NumMatricula) ON DELETE CASCADE,
    CONSTRAINT recibe_asig_fk FOREIGN KEY (CodAsignatura)
        REFERENCES ASIGNATURA (CodAsignatura)
);
```

Ejecutamos

```
Table RECIBE creado.

Nombre          ¿Nulo?   Tipo
--------------- -------- ------------
NUM_MATRICULA   NOT NULL VARCHAR2(10)
COD_ASIGNATURA  NOT NULL VARCHAR2(6)
CURSO_ESCOLAR   NOT NULL VARCHAR2(9)
```

- PRIMARY KEY (campo1, campo2, campo3): Como la clave afecta a varias columnas, se debe definir como restricción de tabla al final.
- ON DELETE CASCADE: Si borramos a un alumno de la tabla ALUMNO, automáticamente se borrarán todas sus filas en la tabla RECIBE para no dejar datos huérfanos.

Es la tabla más compleja porque une a los alumnos con las asignaturas.

- CREATE TABLE RECIBE: "Crea el registro de matriculaciones (quién estudia qué)".

- NumMatricula NUMBER(10), CodAsignatura NUMBER(5), CursoEscolar VARCHAR2(9): "Columnas para anotar quién, en qué materia y en qué año".

- CONSTRAINT recibe_pk PRIMARY KEY (NumMatricula, CodAsignatura, CursoEscolar): "Para que una fila sea única, necesitamos el conjunto de los tres datos. Así evitamos que un alumno se apunte dos veces a la misma asignatura en el mismo año".

- CONSTRAINT rec_alum_fk FOREIGN KEY (NumMatricula) REFERENCES ALUMNO (NumMatricula) ON DELETE CASCADE: "Si un alumno se da de baja de la escuela y lo borramos de la tabla ALUMNO, borra automáticamente todos sus registros de aquí para no dejar basura en el sistema".

- CONSTRAINT rec_asig_fk FOREIGN KEY (CodAsignatura) REFERENCES ASIGNATURA (CodAsignatura): "Asegúrate de que la asignatura exista en la lista oficial de asignaturas".

ENTONCES:

VARCHAR2(n): Texto que se ajusta a lo que escribes.

NUMBER(p,d): Si tuvieras una nota (ej. 9.75), sería NUMBER(4,2) (4 dígitos en total, 2 de ellos decimales).

En la teoría, NUMBER(p, s):

> p (Precisión): Es el número máximo total de dígitos que el campo puede almacenar, contando tanto los que van a la izquierda como los que van a la derecha del punto decimal.
>
> s (Escala): Es el número de dígitos que se reservan exclusivamente para la parte decimal (a la derecha del punto).
>
> *¿Por qué NUMBER(4,2) para una nota de 9.75?*
>
> Vemos que "9.75" solo tiene 3 dígitos, pero cuando diseñamos la base de datos (DDL), no pensamos solo en un ejemplo, sino en el valor más grande posible que queremos que quepa en esa columna.
>
> El caso del 10.00: Si un alumno saca la nota máxima, un 10.00 tiene 4 dígitos en total. Si pusieras NUMBER(3,2), el sistema solo te dejaría guardar números hasta el 9.99. Al intentar meter un 10.00, Oracle te daría un error porque "no cabe" en 3 huecos totales.
>
> Huecos reservados: Al poner NUMBER(4,2), le estás diciendo a Oracle: "Resérvame un espacio total de 4 dígitos, y de esos 4, quiero que 2 sean siempre para los decimales".

DATE: Guarda el día, pero también la hora, minutos y segundos.

Código completo:

```
-- Creamos el usuario con tus iniciales (ejemplo: ASMA1)

CREATE USER C##ASMA1 IDENTIFIED BY "123456";


-- Le damos permisos para que pueda conectarse y crear cosas

GRANT CONNECT, RESOURCE TO C##ASMA1;

-- Cuota ilimitada en el tablespace para que no de error al
insertar datos

ALTER USER C##ASMA1 QUOTA UNLIMITED ON USERS;
```

```
/* EJERCICIO 1. ACTIVIDAD 2 - Diseño Físico */

--1.TABLA ALUMNO
CREATE TABLE ALUMNO (
    NumMatricula NUMBER(10),
    Nombre VARCHAR2(50) NOT NULL,
    FechaNacimiento DATE,
    Telefono NUMBER(9),
    CONSTRAINT alumno_pk PRIMARY KEY (NumMatricula)
);
describe ALUMNO;

--2.TABLA PROFESOR
CREATE TABLE PROFESOR (
    IdProfesor NUMBER(5),
    NIF_P VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Especialidad VARCHAR2(30),
    Telefono NUMBER(9),
    CONSTRAINT profesor_pk PRIMARY KEY (IdProfesor),
    CONSTRAINT profesor_nif_uq UNIQUE (NIF_P)
);

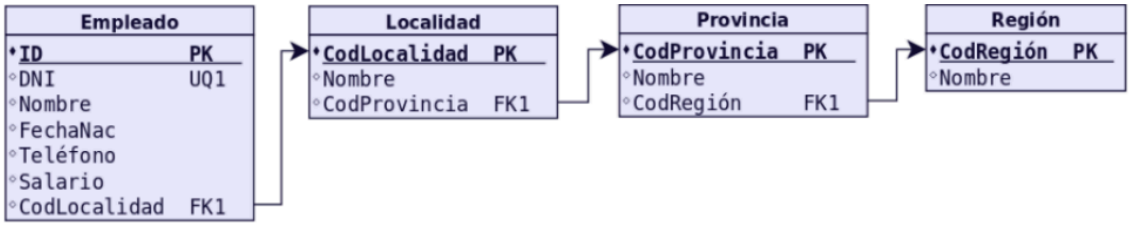
--3. TABLA ASIGNATURA
CREATE TABLE ASIGNATURA (
    CodAsignatura NUMBER(5),
```

```
    Nombre VARCHAR2(40) NOT NULL,
    IdProfesor NUMBER(5),
    CONSTRAINT asignatura_pk PRIMARY KEY (CodAsignatura),
    CONSTRAINT asignatura_prof_fk FOREIGN KEY (IdProfesor)
        REFERENCES PROFESOR (IdProfesor)
);



--4.TABLA RECIBE
CREATE TABLE RECIBE (
    NumMatricula NUMBER(10),
    CodAsignatura NUMBER(5),
    CursoEscolar VARCHAR2(9),
    CONSTRAINT recibe_pk PRIMARY KEY (NumMatricula, CodAsignatura,
CursoEscolar),
    CONSTRAINT recibe_alum_fk FOREIGN KEY (NumMatricula)
        REFERENCES ALUMNO (NumMatricula) ON DELETE CASCADE,
    CONSTRAINT recibe_asig_fk FOREIGN KEY (CodAsignatura)
        REFERENCES ASIGNATURA (CodAsignatura)
);
```

2. Realiza el diseño físico para el siguiente modelo relacional. Asigna el tipo de datos que consideres más adecuado.Esquema C##TUSINICIALESA2, contraseña "123456".



Antes de seguir:

Comando → Qué te muestra
DESCRIBE Empleado; → Nombres de columnas, tipos de datos y si aceptan nulos.
SELECT * FROM Empleado; → Los datos reales que guardaste en la tabla.
SHOW CREATE TABLE Empleado; → El script completo con PK, FK y UQ detalladas.

Empezamos:

Creamos el usuario:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA2 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA2;

ALTER USER C##ASMA2 QUOTA UNLIMITED ON USERS;
```

Lo ejecutamos.

```
Grant correcto.


User C##ASMA2 alterado.
```

OJITO!!!

El Orden de las Tablas: Si intentas crear EMPLEADO primero, Oracle te dirá: *"Error: la tabla LOCALIDAD no existe"*. Por eso siempre empezamos desde la tabla más independiente (REGIÓN) hacia la más dependiente.

Vamos ir paso a paso:

1. El Orden:
No puedes crear una clave foránea (FK) que apunte a una tabla que no existe aún. Por tanto, debemos empezar por "el final" de la cadena:
PRIMERO → Región (no depende de nadie).
SEGUNDO → Provincia (depende de Región).
TERCERO → Localidad (depende de Provincia).
CUARTO →Empleado (depende de Localidad).

Hemos aprendido a,

Definir una Columna

Se escribe: nombre_columna + tipo_dato + opciones.

- **VARCHAR2(n)**: Texto de hasta n caracteres.
- **NUMBER(p, s)**: Números. Para el **Salario**, usaremos NUMBER(8,2) (6 enteros y 2 decimales).
- **DATE**: Para fechas como **FechaNac**.

Las Restricciones (CONSTRAINT)

Aunque se pueden poner en la misma línea del campo, la teoría recomienda ponerlas al final para que el código sea claro:

- **PRIMARY KEY (PK)**: Identificador único. No permite repetidos ni nulos.
- **UNIQUE (UQ)**: Para datos que no son la clave principal pero no deben repetirse, como el **DNI**.
- **FOREIGN KEY (FK)**: Conecta con otra tabla.

★ **Tabla REGIÓN**

```
-- 1. TABLA REGIÓN (La base)
CREATE TABLE REGION (
    CodRegion NUMBER(3),
    Nombre VARCHAR2(50) NOT NULL,
    CONSTRAINT region_pk PRIMARY KEY (CodRegion)
);
```

Es como decir: Crea la tabla Región. El código es la llave maestra (PK). El nombre es obligatorio (NOT NULL).

Ejecutamos

```
Table REGION creado.
```

```
Table REGION creado.

Nombre     ¿Nulo?   Tipo
---------- -------- ------------
CODREGION  NOT NULL VARCHAR2(3)
NOMBRE     NOT NULL VARCHAR2(50)
```

★ **Tabla provincia**

```
--2. TABLA PROVINCIA (Depende de REGION)

CREATE TABLE PROVINCIA (
    CodProvincia NUMBER(3),
    Nombre VARCHAR2(50) NOT NULL,
    CodRegion NUMBER(3),
    CONSTRAINT provincia_pk PRIMARY KEY (CodProvincia),
    CONSTRAINT prov_reg_fk FOREIGN KEY (CodRegion)
        REFERENCES REGION (CodRegion)
);
```

Es como decir: Para crear una provincia, necesito anotar a qué región pertenece. Esa FK asegura que no pongas un código de región que no exista en la tabla anterior.

★ **Tabla localidad**

```
CREATE TABLE LOCALIDAD (
    CodLocalidad NUMBER(5),
    Nombre VARCHAR2(50) NOT NULL,
    CodProvincia NUMBER(3),
    CONSTRAINT localidad_pk PRIMARY KEY (CodLocalidad),
    CONSTRAINT loc_prov_fk FOREIGN KEY (CodProvincia)
        REFERENCES PROVINCIA (CodProvincia)
);
```

- **CREATE TABLE LOCALIDAD (**: Vamos a crear un registro oficial de todos los pueblos y ciudades".
- **CodLocalidad NUMBER(5),**: Cada localidad tendrá un código numérico de hasta 5 dígitos (por ejemplo, el 41001)".
- **Nombre VARCHAR2(50) NOT NULL,**: "Guardaremos el nombre de la localidad. Hemos reservado hasta 50 letras y es **obligatorio** ponerlo; no puede haber una localidad sin nombre.
- **CodProvincia NUMBER(3),**: "Aquí anotaremos el número de la provincia a la que pertenece este pueblo.
- **CONSTRAINT localidad_pk PRIMARY KEY (CodLocalidad),**: El código de la localidad es su DNI único. No puede haber dos pueblos con el mismo código en nuestra base de datos".
- **CONSTRAINT loc_prov_fk FOREIGN KEY (CodProvincia) REFERENCES PROVINCIA (CodProvincia)**: El número de provincia que escribas aquí **tiene que existir** primero en la tabla de Provincias. No puedes decir que un pueblo es de la provincia '99' si esa provincia no está creada.

★ **Tabla empleado**

```
CREATE TABLE EMPLEADO (
    ID NUMBER(10),
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    FechaNac DATE,
    Telefono NUMBER(9),
    Salario NUMBER(8,2), -- Hasta 999.999,99
    CodLocalidad NUMBER(5),
    CONSTRAINT empleado_pk PRIMARY KEY (ID),
    CONSTRAINT empleado_dni_uq UNIQUE (DNI),
    CONSTRAINT emp_loc_fk FOREIGN KEY (CodLocalidad)
        REFERENCES LOCALIDAD (CodLocalidad)
);
```

- **CREATE TABLE EMPLEADO (**: "Creamos la ficha maestra de nuestros trabajadores".
- **ID NUMBER(10),**: "Un número de identificación interno de hasta 10 cifras".
- **DNI VARCHAR2(9),**: "Guardaremos su DNI (8 números y la letra, por eso usamos VARCHAR2)".
- **Nombre VARCHAR2(50) NOT NULL,**: "Su nombre y apellidos. Es obligatorio rellenarlo".
- **FechaNac DATE,**: "Su fecha de nacimiento en formato día/mes/año".
- **Telefono NUMBER(9),**: "Su número de teléfono (los 9 dígitos estándar)".

- **Salario NUMBER(8,2),**: "Aquí guardamos lo que gana. El '8' significa que el número puede tener **8 cifras en total**, y el '2' significa que **dos de ellas son para los céntimos**. (El sueldo máximo que cabría sería 999.999,99)".
- **CodLocalidad NUMBER(5),**: "Aquí apuntamos el código del pueblo donde vive el empleado".
- **CONSTRAINT empleado_pk PRIMARY KEY (ID),**: "El campo ID es la llave maestra. Es lo que usaremos para buscar al empleado sin confundirlo con otro".
- **CONSTRAINT empleado_dni_uq UNIQUE (DNI),**: "Aunque el ID es la llave, el DNI también es único en el mundo real. Esta regla prohíbe que metas por error a dos empleados con el mismo DNI".
- **CONSTRAINT emp_loc_fk FOREIGN KEY (CodLocalidad) REFERENCES LOCALIDAD (CodLocalidad)**: "Esta es la conexión final: el código de localidad del empleado debe ser uno de los que ya tenemos registrados en nuestra lista de localidades".

codigo completo :

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA2 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA2;

ALTER USER C##ASMA2 QUOTA UNLIMITED ON USERS;

-- 1. TABLA REGIÓN (La base)
CREATE TABLE REGION (
    CodRegion NUMBER(3),
    Nombre VARCHAR2(50) NOT NULL,
    CONSTRAINT region_pk PRIMARY KEY (CodRegion)
);

--2. TABLA PROVINCIA (Depende de REGION)

CREATE TABLE PROVINCIA (
    CodProvincia NUMBER(3),
    Nombre VARCHAR2(50) NOT NULL,
    CodRegion NUMBER(3),
    CONSTRAINT provincia_pk PRIMARY KEY (CodProvincia),
    CONSTRAINT prov_reg_fk FOREIGN KEY (CodRegion)
        REFERENCES REGION (CodRegion)
);

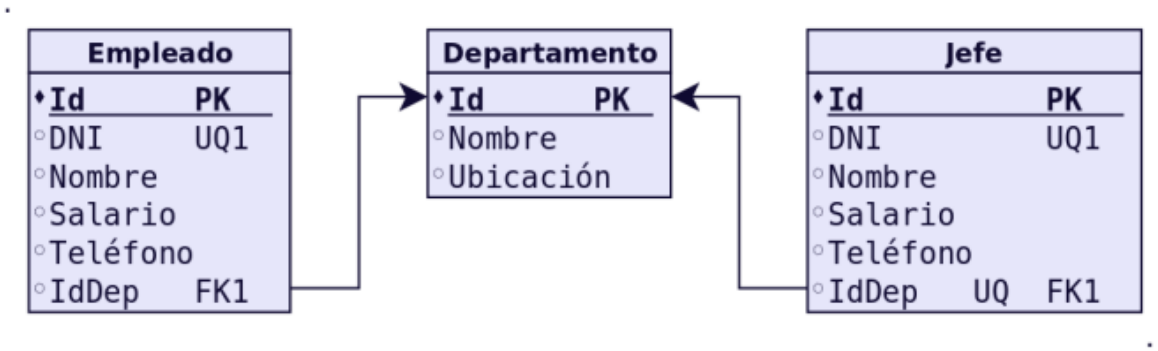

--3. TABLA LOCALIDAD (Depende de PROVINCIA)
```

```
CREATE TABLE LOCALIDAD (
    CodLocalidad NUMBER(5),
    Nombre VARCHAR2(50) NOT NULL,
    CodProvincia NUMBER(3),
    CONSTRAINT localidad_pk PRIMARY KEY (CodLocalidad),
    CONSTRAINT loc_prov_fk FOREIGN KEY (CodProvincia)
        REFERENCES PROVINCIA (CodProvincia)
);

--4. TABLA EMPELADO (Depende de LOCALIDAD)

CREATE TABLE EMPLEADO (
    ID NUMBER(10),
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    FechaNac DATE,
    Telefono NUMBER(9),
    Salario NUMBER(8,2), -- Hasta 999.999,99
    CodLocalidad NUMBER(5),
    CONSTRAINT empleado_pk PRIMARY KEY (ID),
    CONSTRAINT empleado_dni_uq UNIQUE (DNI),
    CONSTRAINT emp_loc_fk FOREIGN KEY (CodLocalidad)
        REFERENCES LOCALIDAD (CodLocalidad)
);
```

3. Realiza el diseño físico para el siguiente modelo relacional. Asigna el tipo de datos que consideres más adecuado. Esquema C##TUSINICIALESA3, contraseña "123456"..



Como siempre, miramos quién depende de quién:

- Departamento: Es la tabla maestra en este caso. No tiene flechas que salgan de ella hacia otros.
- Empleado: Depende de Departamento.

- Jefe: También depende de Departamento.

Creamos el usuario:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA3 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA3;

ALTER USER C##ASMA3 QUOTA UNLIMITED ON USERS;
```

★ **TABLA DEPARTAMENTO**

```
-- 1. TABLA DEPARTAMENTO
CREATE TABLE DEPARTAMENTO (
    Id NUMBER(5),
    Nombre VARCHAR2(50) NOT NULL,
    Ubicacion VARCHAR2(50),
    CONSTRAINT departamento_pk PRIMARY KEY (Id)
);
DESC DEPARTAMENTO;
```

Es como decir: Crea la oficina central. El código Id es la llave que abre esta tabla (PK). El nombre es obligatorio porque no existen departamentos sin nombre.

★ **TABLA EMPLEADO**

```
-- 2. TABLA EMPLEADO
CREATE TABLE EMPLEADO (
    Id NUMBER(10),
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Salario NUMBER(8,2),
    Telefono NUMBER(9),
    IdDep NUMBER(5),
    CONSTRAINT empleado_pk PRIMARY KEY (Id),
    CONSTRAINT empleado_dni_uq UNIQUE (DNI),
    CONSTRAINT emp_dep_fk FOREIGN KEY (IdDep)
        REFERENCES DEPARTAMENTO (Id)
);
DESC EMPLEADO;
```

Es como decir: Crea la lista de trabajadores. El `Id` es su identificador único. El `Salario` tiene 8 dígitos totales con 2 decimales (hasta 999.999,99). La `FK` dice: 'Para meter a este

empleado, dime en qué departamento trabaja, y ese departamento tiene que existir en la lista anterior.

★ **TABLA JEFE**

```
--3.TABLA JEFE
-- En este modelo, el Jefe se define de forma separada al empleado.
CREATE TABLE JEFE (
    Id NUMBER(10),
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Salario NUMBER(8,2),
    Telefono NUMBER(9),
    IdDep NUMBER(5),
    CONSTRAINT jefe_pk PRIMARY KEY (Id),
    CONSTRAINT jefe_dni_uq UNIQUE (DNI),
    CONSTRAINT jefe_dep_uq UNIQUE (IdDep), -- ESTA ES LA CLAVE
    CONSTRAINT jefe_dep_fk FOREIGN KEY (IdDep)
        REFERENCES DEPARTAMENTO (Id)
);
DESC JEFE;
```

Es como decir: Crea la lista de jefes. Es casi igual a la de empleados, pero ojo con UNIQUE (IdDep). Esto significa que solo puede haber un jefe por cada departamento. Si intentas asignar un segundo jefe al mismo departamento, Oracle te dirá que 'nones' porque el código de departamento no se puede repetir en esta lista.

CODIGO COMPLETO

```
/* EJERCICIO 3: Relaciones 1:N y 1:1 */

-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA3 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA3;

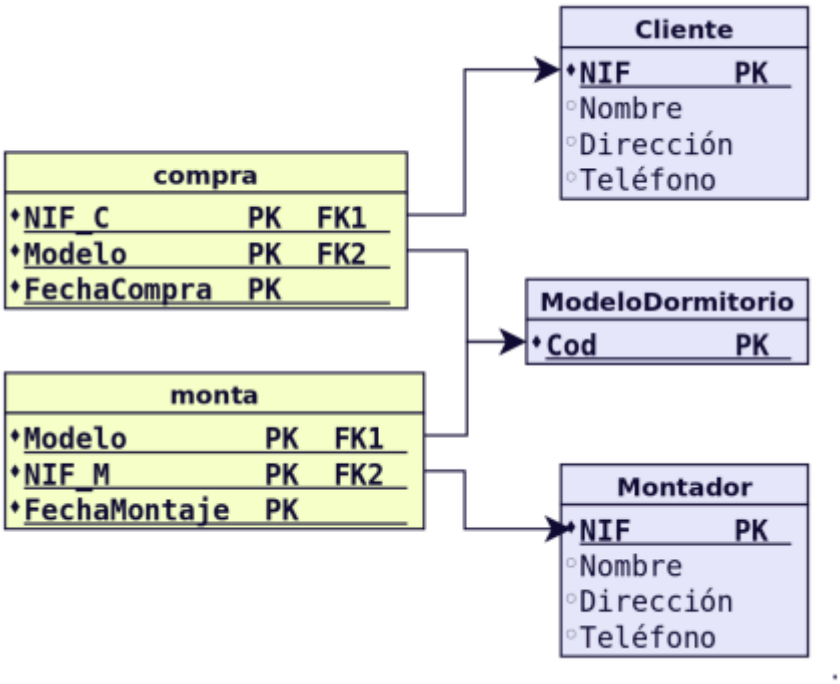
ALTER USER C##ASMA3 QUOTA UNLIMITED ON USERS;

-- 1. TABLA DEPARTAMENTO
CREATE TABLE DEPARTAMENTO (
    Id NUMBER(5),
    Nombre VARCHAR2(50) NOT NULL,
    Ubicacion VARCHAR2(50),
    CONSTRAINT departamento_pk PRIMARY KEY (Id)
);
DESC DEPARTAMENTO;
-- 2. TABLA EMPLEADO
```

```
CREATE TABLE EMPLEADO (
    Id NUMBER(10),
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Salario NUMBER(8,2),
    Telefono NUMBER(9),
    IdDep NUMBER(5),
    CONSTRAINT empleado_pk PRIMARY KEY (Id),
    CONSTRAINT empleado_dni_uq UNIQUE (DNI),
    CONSTRAINT emp_dep_fk FOREIGN KEY (IdDep)
        REFERENCES DEPARTAMENTO (Id)
);
DESC EMPLEADO;

--3.TABLA JEFE
-- En este modelo, el Jefe se define de forma separada al
empleado.
CREATE TABLE JEFE (
    Id NUMBER(10),
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Salario NUMBER(8,2),
    Telefono NUMBER(9),
    IdDep NUMBER(5),
    CONSTRAINT jefe_pk PRIMARY KEY (Id),
    CONSTRAINT jefe_dni_uq UNIQUE (DNI),
    CONSTRAINT jefe_dep_uq UNIQUE (IdDep), -- ESTA ES LA CLAVE
    CONSTRAINT jefe_dep_fk FOREIGN KEY (IdDep)
        REFERENCES DEPARTAMENTO (Id)
);
DESC JEFE;
```

4. Realiza el diseño físico para el siguiente modelo relacional. Asigna el tipo de datos que consideres más adecuado.Esquema C##TUSINICIALESA4, contraseña “123456”.



Creamos el usuario:

```
-- CREACION DEL USUARIO
--
CREATE USER C##ASMA4 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA4;

ALTER USER C##ASMA4 QUOTA UNLIMITED ON USERS;
```

Primero creamos las tablas maestras (las que no tienen FK) y luego las de relación:

Cliente, ModeloDormitorio y Montador.

Compra (depende de Cliente y ModeloDormitorio).

Monta (depende de ModeloDormitorio y Montador).

★ **TABLA CLIENTE**

```
--TABLAS INDEPENDIENTES

--TABLA CLIENTE
CREATE TABLE Cliente (
    NIF VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    CONSTRAINT cliente_pk PRIMARY KEY (NIF)
);
```

★ **TABLA MODELODORMITORIO**

```
--TABLA MODELODORMITORIO
CREATE TABLE ModeloDormitorio (
    Cod NUMBER(5),
    CONSTRAINT modelo_pk PRIMARY KEY (Cod)
);
```

★ **TABLA MONTADOR**

```
--TABLA MONTADOR
CREATE TABLE Montador (
    NIF VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    CONSTRAINT montador_pk PRIMARY KEY (NIF)
);
```

Es como decir: Creamos las tres listas básicas: la de clientes, la de tipos de muebles y la de los trabajadores que los montan. En cada una, el NIF o el Código sirven como llave maestra (PK) para que no haya confusiones.

Tablas de Relación (Las que unen)

Aquí es donde aplicamos la restricción de tabla para las claves primarias compuestas.

★ **TABLA COMPRA**

Compra depende de compra y modelodormitorio.

```
--TABLA COMPRA

CREATE TABLE compra (
    NIF_C VARCHAR2(9),
    Modelo NUMBER(5),
    FechaCompra DATE,
    CONSTRAINT compra_pk PRIMARY KEY (NIF_C, Modelo, FechaCompra),
    CONSTRAINT compra_cli_fk FOREIGN KEY (NIF_C) REFERENCES Cliente (NIF),
    CONSTRAINT compra_mod_fk FOREIGN KEY (Modelo) REFERENCES ModeloDormitorio (Cod)
);
```

Aquí anotamos qué cliente compró qué mueble y cuándo. Como un cliente puede comprar muchos muebles y un mueble ser comprado por muchos clientes, necesitamos que la 'identidad' de la fila (PK) sea la combinación de los tres campos. Además, las FK vigilan que tanto el cliente como el mueble existan de verdad en sus listas.

★ **TABLA MONTA**

Monta depende de modelodormitorio y montador.

```
--TABLA MONTA

CREATE TABLE monta (
    Modelo NUMBER(5),
    NIF_M VARCHAR2(9),
    FechaMontaje DATE,
    CONSTRAINT monta_pk PRIMARY KEY (Modelo, NIF_M, FechaMontaje),
    CONSTRAINT monta_mod_fk FOREIGN KEY (Modelo) REFERENCES ModeloDormitorio (Cod),
    CONSTRAINT monta_mon_fk FOREIGN KEY (NIF_M) REFERENCES Montador (NIF)
);
```

Igual que la anterior, pero para registrar qué montador se encargó de cada dormitorio. La clave primaria es el conjunto de los tres datos para que quede registrado cada montaje individual.

ENTONCES:

- Clave Primaria Compuesta: Se usa cuando un solo campo no basta para identificar la fila. Se debe definir siempre al final con CONSTRAINT ... PRIMARY KEY (campo1, campo2...).

- Integridad Referencial: Las FK impiden que borres un mueble si hay una compra registrada, a menos que uses ON DELETE CASCADE.

- VARCHAR2 vs NUMBER: Usamos VARCHAR2(9) para el NIF porque contiene una letra, y NUMBER(9) para el teléfono porque son solo dígitos.

Código completo:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA4 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA4;

ALTER USER C##ASMA4 QUOTA UNLIMITED ON USERS;

--TABLAS INDEPENDIENTES

--TABLA CLIENTE
CREATE TABLE Cliente (
    NIF VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    CONSTRAINT cliente_pk PRIMARY KEY (NIF)
);

--TABLA MODELODORMITORIO
CREATE TABLE ModeloDormitorio (
    Cod NUMBER(5),
    CONSTRAINT modelo_pk PRIMARY KEY (Cod)
);


--TABLA MONTADOR
CREATE TABLE Montador (
    NIF VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    CONSTRAINT montador_pk PRIMARY KEY (NIF)
```

```
);


--TABLA COMPRA

CREATE TABLE compra (
    NIF_C VARCHAR2(9),
    Modelo NUMBER(5),
    FechaCompra DATE,
    CONSTRAINT compra_pk PRIMARY KEY (NIF_C, Modelo, FechaCompra),
    CONSTRAINT compra_cli_fk FOREIGN KEY (NIF_C) REFERENCES
Cliente (NIF),
    CONSTRAINT compra_mod_fk FOREIGN KEY (Modelo) REFERENCES
ModeloDormitorio (Cod)
);

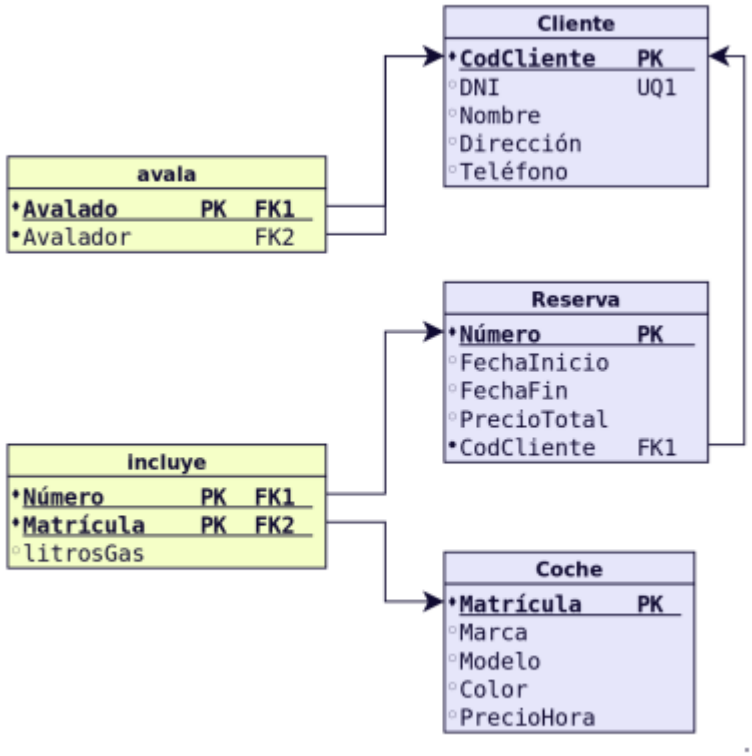
--TABLA MONTA

CREATE TABLE monta (
    Modelo NUMBER(5),
    NIF_M VARCHAR2(9),
    FechaMontaje DATE,
    CONSTRAINT monta_pk PRIMARY KEY (Modelo, NIF_M, FechaMontaje),
    CONSTRAINT monta_mod_fk FOREIGN KEY (Modelo) REFERENCES
ModeloDormitorio (Cod),
    CONSTRAINT monta_mon_fk FOREIGN KEY (NIF_M) REFERENCES
Montador (NIF)
);
```

5. Realiza el diseño físico para el siguiente modelo relacional. Asigna el tipo de datos que

consideres más adecuado.Esquema C##TUSINICIALESA5, contraseña “123456”. No pueden ser nulos los siguientes campos: Nombre de Cliente, Marca y Modelo de Coche.

Crear una secuencia para el Número de Reserva.



Creamos el usuario:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA5 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA5;

ALTER USER C##ASMA5 QUOTA UNLIMITED ON USERS;
```

Alquiler de Coches. Este modelo tiene algo especial: una relación "recursiva" (un cliente avala a otro) y el uso de una **secuencia**.

- Primero, ordenamos:

1. **Coche** y **Cliente** (Son las independientes).
2. **Avala** (Depende de Cliente).
3. **Reserva** (Depende de Cliente).
4. **Incluye** (Depende de Reserva y Coche).

- Segundo, creamos la SECUENCIA

  Antes de las tablas, creamos la herramienta para los números de reserva.

```
--Creamos la SECUENCIA
CREATE SEQUENCE seq_num_reserva START WITH 1 INCREMENT BY 1;
```

Salida de Script

Tarea terminada en 0,061 segundos

```
Sequence SEQ_NUM_RESERVA creado.
```

Es como crear un contador automático llamado seq_num_reserva. Empezará en 1 y sumará de uno en uno cada vez que alguien pida un número nuevo.

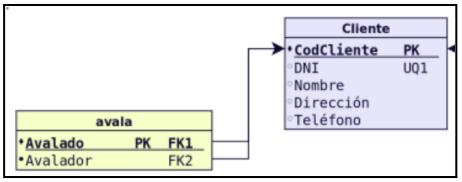
- Tercero, creamos las tablas (en orden):

★ **TABLA COCHE**

```
--TABLA COCHE
CREATE TABLE Coche (
    Matricula VARCHAR2(8),
    Marca VARCHAR2(20) NOT NULL,
    Modelo VARCHAR2(20) NOT NULL,
    Color VARCHAR2(15),
    PrecioHora NUMBER(5,2),
    CONSTRAINT coche_pk PRIMARY KEY (Matricula)
);
```

Es como decir, crea la lista de coches. La matrícula es la llave maestra (PK). Es obligatorio poner la marca y el modelo (NOT NULL). El precio por hora admite números como 15.50 (5 dígitos totales, 2 decimales)

★ **TABLA CLIENTE (INCLUYE EL AVAL)**




```
--TABLA CLIENTE(INCLUYE EL AVAL)
CREATE TABLE Cliente (
    CodCliente NUMBER(6),
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    CONSTRAINT cliente_pk PRIMARY KEY (CodCliente),
    CONSTRAINT cliente_dni_uq UNIQUE (DNI)
);
```

→ Lista de clientes. El código es la PK. El DNI es único (UQ1). El nombre es obligatorio por contrato.

★ **TABLA AVALA (TABLA RECURSIVA)**

Aquí es donde un cliente avala a otro.

```
--TABLA AVALA
CREATE TABLE avala (
    Avalado NUMBER(6),
    Avalador NUMBER(6),
    CONSTRAINT avala_pk PRIMARY KEY (Avalado),
    CONSTRAINT avala_cli_fk1 FOREIGN KEY (Avalado) REFERENCES Cliente (CodCliente),
    CONSTRAINT avala_cli_fk2 FOREIGN KEY (Avalador) REFERENCES Cliente (CodCliente)
);
```

→Anotamos quién avala a quién. La PK es el Avalado (porque según el esquema, un cliente solo tiene un avalador principal). Ambas columnas deben ser clientes que ya existan en la tabla anterior.

★ **TABLA RESERVA**

Aquí usaremos la secuencia que creamos. →

```
--Creamos la SECUENCIA
CREATE SEQUENCE seq_num_reserva START WITH 1 INCREMENT BY 1;
```

Salida de Script

Tarea terminada en 0,061 segundos

```
Sequence SEQ_NUM_RESERVA creado.
```

Es como crear un contador automático llamado seq_num_reserva. Empezará en 1 y sumará de uno en uno cada vez que alguien pida un número nuevo.

En la teoría de Oracle, una secuencia es un objeto independiente de la tabla.

Pero….¿Por qué no aparece en el CREATE TABLE? → Cuando diseñas la tabla Reserva, solo defines que el campo Numero es de tipo NUMBER. La tabla no sabe de dónde vendrá el número; solo sabe que debe recibir uno que sea único.

Y……¿Cuándo y cómo se usa? (El momento del INSERT) → Usamos la secuencia en el momento de introducir los datos (DML). En lugar de escribir nosotros el número "1", "2" o "3", le pedimos a la secuencia su "Siguiente Valor" usando la palabra clave .NEXTVAL.

Es como dijeramos: “Inserta una nueva reserva, y para el campo Numero, ve al dispensador seq_num_reserva y dame el siguiente valor que toque”

Aún no hemos llegado a los INSERT INTO, pero sería así:

```
INSERT INTO Reserva (Numero, FechaInicio, CodCliente)

VALUES (seq_num_reserva.NEXTVAL, '2026-02-01', 101);

--       …….

--       Esto saca el ticket automático
```

Entonces:

- CREATE SEQUENCE: Fabricamos el dispensador de números.
- CREATE TABLE: Preparamos la carpeta donde guardaremos los papeles con esos números.
- INSERT con .NEXTVAL: Es el momento real en el que el dispensador le da el número a la tabla.

```
--TABLA RESERVA

CREATE TABLE Reserva (
    Numero NUMBER(8),
    FechaInicio DATE,
    FechaFin DATE,
    PrecioTotal NUMBER(8,2),
    CodCliente NUMBER(6),
    CONSTRAINT reserva_pk PRIMARY KEY (Numero),
    CONSTRAINT res_cli_fk FOREIGN KEY (CodCliente) REFERENCES Cliente (CodCliente)
);
```

→ Registro de reservas. El Numero será la PK. Apuntamos fechas de inicio y fin (DATE). El PrecioTotal permite hasta 999.999,99. Cada reserva debe estar a nombre de un cliente real

★ **TABLA INCLUYE**

```
--TABLA INCLUYE
CREATE TABLE incluye (
    Numero NUMBER(8),
    Matricula VARCHAR2(8),
    litrosGas NUMBER(5,2),
    CONSTRAINT incluye_pk PRIMARY KEY (Numero, Matricula),
    CONSTRAINT inc_res_fk FOREIGN KEY (Numero) REFERENCES Reserva (Numero),
    CONSTRAINT inc_coc_fk FOREIGN KEY (Matricula) REFERENCES Coche (Matricula)
);
```

→ Detalle de qué coches van en cada reserva. La PK es la pareja Numero y Matricula para que no se repita el mismo coche en la misma reserva. Los litros de gasolina admiten decimales

Código completo:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA5 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA5;

ALTER USER C##ASMA5 QUOTA UNLIMITED ON USERS;

--Creamos la SECUENCIA
CREATE SEQUENCE seq_num_reserva START WITH 1 INCREMENT BY 1;

--TABLA COCHE
CREATE TABLE Coche (
    Matricula VARCHAR2(8),
    Marca VARCHAR2(20) NOT NULL,
    Modelo VARCHAR2(20) NOT NULL,
    Color VARCHAR2(15),
    PrecioHora NUMBER(5,2),
    CONSTRAINT coche_pk PRIMARY KEY (Matricula)
);

--TABLA CLIENTE(INCLUYE EL AVAL)
CREATE TABLE Cliente (
    CodCliente NUMBER(6),
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    CONSTRAINT cliente_pk PRIMARY KEY (CodCliente),
    CONSTRAINT cliente_dni_uq UNIQUE (DNI)
);

--TABLA AVALA
CREATE TABLE avala (
    Avalado NUMBER(6),
    Avalador NUMBER(6),
    CONSTRAINT avala_pk PRIMARY KEY (Avalado),
    CONSTRAINT avala_cli_fk1 FOREIGN KEY (Avalado) REFERENCES
Cliente (CodCliente),
    CONSTRAINT avala_cli_fk2 FOREIGN KEY (Avalador) REFERENCES
Cliente (CodCliente)
);

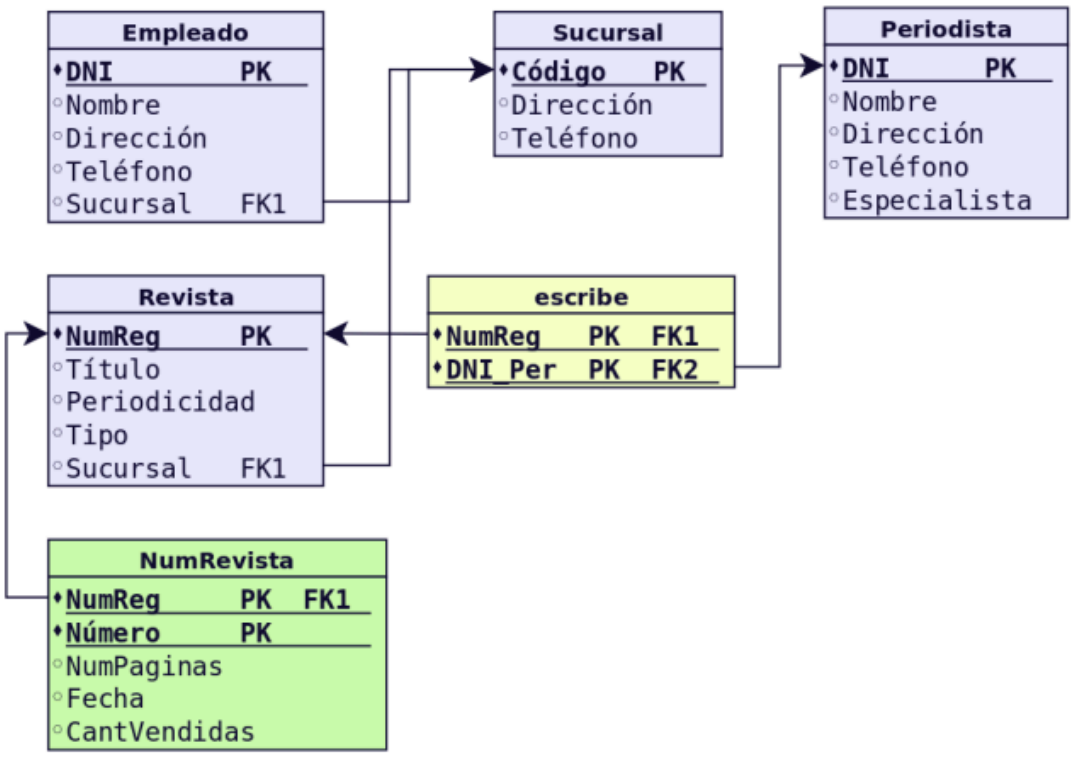

--TABLA RESERVA

CREATE TABLE Reserva (
```

```
    Numero NUMBER(8),
    FechaInicio DATE,
    FechaFin DATE,
    PrecioTotal NUMBER(8,2),
    CodCliente NUMBER(6),
    CONSTRAINT reserva_pk PRIMARY KEY (Numero),
    CONSTRAINT res_cli_fk FOREIGN KEY (CodCliente) REFERENCES
Cliente (CodCliente)
);


--TABLA INCLUYE
CREATE TABLE incluye (
    Numero NUMBER(8),
    Matricula VARCHAR2(8),
    litrosGas NUMBER(5,2),
    CONSTRAINT incluye_pk PRIMARY KEY (Numero, Matricula),
    CONSTRAINT inc_res_fk FOREIGN KEY (Numero) REFERENCES Reserva
(Numero),
    CONSTRAINT inc_coc_fk FOREIGN KEY (Matricula) REFERENCES Coche
(Matricula)
);
```

6. Realiza el diseño físico para el siguiente modelo relacional. Asigna el tipo de datos que consideres más adecuado. Realiza el diseño poniendo nombres a las restricciones. Esquema C##TUSINICIALESA6, contraseña "123456". No pueden ser nulos los siguientes campos: Nombre de Empleado, Nombre de Periodista, Título de Revista. La Periodicidad toma uno de los siguientes valores: Semanal, Quincenal, Mensual, Trimestral o Anual, siendo el valor por defecto Mensual. NumPaginas debe ser mayor que 0.



→ Gestión de **Revistas y Sucursales**

Primero, creamos el usuario:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA6 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA6;

ALTER USER C##ASMA6 QUOTA UNLIMITED ON USERS;
```

Segundo, ordenamos:

1. **Sucursal** y **Periodista** (No dependen de nadie).
2. **Empleado** y **Revista** (Ambas dependen de Sucursal).
3. **escribe** (Relación entre Revista y Periodista).
4. **NumRevista** (Entidad débil que depende de Revista).

★ **TABLA SUCURSAL**

```
--TABLA SUCURSAL

CREATE TABLE Sucursal (
    Codigo NUMBER(5),
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    CONSTRAINT sucursal_pk PRIMARY KEY (Codigo)
);
```

→ Crea la lista de oficinas. El Codigo es su identificador único (PK). Guardamos su dirección y un teléfono de 9 cifras.

★ **TABLA PERIODISTA**

```
--TABLA PERIODISTA
CREATE TABLE Periodista (
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    Especialista VARCHAR2(30),
    CONSTRAINT periodista_pk PRIMARY KEY (DNI)
);
```

→ Crea la ficha de periodistas. El DNI es la llave maestra. Como pide el ejercicio, el Nombre es obligatorio (NOT NULL).

★ **TABLA EMPLEADO**

```
--TABLA EMPLEADO

CREATE TABLE Empleado (
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    Sucursal NUMBER(5),
    CONSTRAINT empleado_pk PRIMARY KEY (DNI),
    CONSTRAINT emp_suc_fk FOREIGN KEY (Sucursal)
        REFERENCES Sucursal (Codigo)
);
```

→ Lista de empleados. El Nombre es obligatorio. Cada empleado debe estar vinculado a una oficina real mediante la FK de Sucursal.

★ **TABLA Revista (Aquí aplicamos los CHECK y DEFAULT)**

```
--TABLA REVISTA
CREATE TABLE Revista (
    NumReg NUMBER(8),
    Titulo VARCHAR2(100) NOT NULL,
    Periodicidad VARCHAR2(15) DEFAULT 'Mensual',
    Tipo VARCHAR2(30),
    Sucursal NUMBER(5),
    CONSTRAINT revista_pk PRIMARY KEY (NumReg),
    CONSTRAINT rev_suc_fk FOREIGN KEY (Sucursal)
        REFERENCES Sucursal (Codigo),
    CONSTRAINT rev_per_ck CHECK (Periodicidad IN ('Semanal', 'Quincenal', 'Mensual', 'Trimestral', 'Anual'))
);
```

→ Crea la tabla de revistas. El Titulo es obligatorio. Si no decimos nada, la Periodicidad será 'Mensual' por defecto. Además, con el CHECK, Oracle vigilará que solo escribas uno de los 5 periodos permitidos.

★ **TABLA ESCRIBE**

```
--TABLA ESCRIBE

CREATE TABLE escribe (
    NumReg NUMBER(8),
    DNI_Per VARCHAR2(9),
    CONSTRAINT escribe_pk PRIMARY KEY (NumReg, DNI_Per),
    CONSTRAINT esc_rev_fk FOREIGN KEY (NumReg) REFERENCES Revista (NumReg),
    CONSTRAINT esc_per_fk FOREIGN KEY (DNI_Per) REFERENCES Periodista (DNI)
);
```

Las tablas de relación (como escribe) son las que más cuestan al principio porque no representan un "objeto" físico (como un coche o una persona), sino una acción o un vínculo entre dos cosas.

Un Periodista puede escribir en muchas Revistas, y una Revista tiene muchos Periodistas trabajando en ella.
Si pusiéramos el nombre del periodista en la tabla Revista, solo podría haber uno por revista.
Si pusiéramos el número de la revista en la tabla Periodista, el periodista solo podría trabajar en una revista.
Bueno, pues…no nos cabe otra que crear una tabla intermedia llamada escribe que sirve como un "registro de firmas".

Entonces:

```
--TABLA ESCRIBE

CREATE TABLE escribe (
    NumReg NUMBER(8),
    DNI_Per VARCHAR2(9),
    CONSTRAINT escribe_pk PRIMARY KEY (NumReg, DNI_Per),
    CONSTRAINT esc_rev_fk FOREIGN KEY (NumReg) REFERENCES Revista (NumReg),
    CONSTRAINT esc_per_fk FOREIGN KEY (DNI_Per) REFERENCES Periodista (DNI)
);
```

- NumReg: Es el número de registro de la revista. Lo sacamos de la tabla Revista.
- DNI_Per: Es el DNI del periodista. Lo sacamos de la tabla Periodista.

```
--TABLA ESCRIBE

CREATE TABLE escribe (
    NumReg NUMBER(8),
    DNI_Per VARCHAR2(9),
    CONSTRAINT escribe_pk PRIMARY KEY (NumReg, DNI_Per),
    CONSTRAINT esc_rev_fk FOREIGN KEY (NumReg) REFERENCES Revista (NumReg),
    CONSTRAINT esc_per_fk FOREIGN KEY (DNI_Per) REFERENCES Periodista (DNI)
);
```

- PRIMARY KEY (NumReg, DNI_Per): Esta es una Clave Primaria Compuesta. Significa que la combinación de ambos campos es lo que identifica la fila de forma única.

No puedes anotar dos veces que el Periodista 'A' escribe en la Revista '1'. Una vez anotado, ese vínculo ya existe y es único.

```
--TABLA ESCRIBE

CREATE TABLE escribe (
    NumReg NUMBER(8),
    DNI_Per VARCHAR2(9),
    CONSTRAINT escribe_pk PRIMARY KEY (NumReg, DNI_Per),
    CONSTRAINT esc_rev_fk FOREIGN KEY (NumReg) REFERENCES Revista (NumReg),
    CONSTRAINT esc_per_fk FOREIGN KEY (DNI_Per) REFERENCES Periodista (DNI)
);
```

- FOREIGN KEY (...) REFERENCES ...: Son los "anclajes" de seguridad (Claves Foráneas).

Es como si dijéramos: Vigila que nadie invente datos. Si anotas que alguien escribe en la revista 500, esa revista 500 debe existir en la tabla Revista. Y si pones un DNI, ese periodista debe estar dado de alta en la tabla Periodista.

Es una tabla de relación (N a M), tiene dos claves foráneas que, juntas, forman la clave primaria. Sirve para saber qué periodistas están asignados a qué revistas sin repetir datos innecesarios.

★ **Tabla NumRevista (La entidad débil)**

```
--TABLA NUMREVISTA
CREATE TABLE NumRevista (
    NumReg NUMBER(8),
    Numero NUMBER(4),
    NumPaginas NUMBER(4),
    Fecha DATE,
    CantVendidas NUMBER(8),
    CONSTRAINT numrevista_pk PRIMARY KEY (NumReg, Numero),
    CONSTRAINT numrev_rev_fk FOREIGN KEY (NumReg) REFERENCES Revista (NumReg),
    CONSTRAINT numrev_pag_ck CHECK (NumPaginas > 0)
);
```

→ Aquí anotamos cada número concreto de una revista. La PK es compuesta (Nº de Registro + Nº de edición). Ponemos un CHECK para que el número de páginas sea siempre positivo.

Hemos visto en este modelo:

- DEFAULT 'Mensual': Si al insertar una revista no ponemos nada, Oracle rellena el campo automáticamente.

- CHECK (columna IN (...)): Sirve para crear una lista cerrada de valores permitidos.

- CHECK (NumPaginas > 0): Asegura que los datos numéricos tengan sentido lógico.

Código:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA6 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA6;

ALTER USER C##ASMA6 QUOTA UNLIMITED ON USERS;

--TABLA SUCURSAL

CREATE TABLE Sucursal (
    Codigo NUMBER(5),
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    CONSTRAINT sucursal_pk PRIMARY KEY (Codigo)
);


--TABLA PERIODISTA
CREATE TABLE Periodista (
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    Especialista VARCHAR2(30),
    CONSTRAINT periodista_pk PRIMARY KEY (DNI)
);


--TABLA EMPLEADO

CREATE TABLE Empleado (
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    Sucursal NUMBER(5),
    CONSTRAINT empleado_pk PRIMARY KEY (DNI),
    CONSTRAINT emp_suc_fk FOREIGN KEY (Sucursal)
```

```
        REFERENCES Sucursal (Codigo)
);

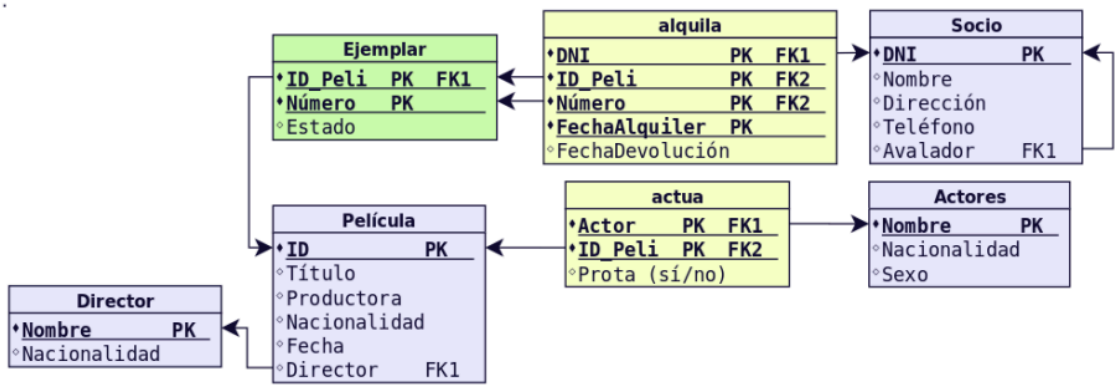
--TABLA REVISTA
CREATE TABLE Revista (
    NumReg NUMBER(8),
    Titulo VARCHAR2(100) NOT NULL,
    Periodicidad VARCHAR2(15) DEFAULT 'Mensual',
    Tipo VARCHAR2(30),
    Sucursal NUMBER(5),
    CONSTRAINT revista_pk PRIMARY KEY (NumReg),
    CONSTRAINT rev_suc_fk FOREIGN KEY (Sucursal)
        REFERENCES Sucursal (Codigo),
    CONSTRAINT rev_per_ck CHECK (Periodicidad IN ('Semanal',
'Quincenal', 'Mensual', 'Trimestral', 'Anual'))
);


--TABLA ESCRIBE

CREATE TABLE escribe (
    NumReg NUMBER(8),
    DNI_Per VARCHAR2(9),
    CONSTRAINT escribe_pk PRIMARY KEY (NumReg, DNI_Per),
    CONSTRAINT esc_rev_fk FOREIGN KEY (NumReg) REFERENCES Revista
(NumReg),
    CONSTRAINT esc_per_fk FOREIGN KEY (DNI_Per) REFERENCES
Periodista (DNI)
);

--TABLA NUMREVISTA
CREATE TABLE NumRevista (
    NumReg NUMBER(8),
    Numero NUMBER(4),
    NumPaginas NUMBER(4),
    Fecha DATE,
    CantVendidas NUMBER(8),
    CONSTRAINT numrevista_pk PRIMARY KEY (NumReg, Numero),
    CONSTRAINT numrev_rev_fk FOREIGN KEY (NumReg) REFERENCES
Revista (NumReg),
    CONSTRAINT numrev_pag_ck CHECK (NumPaginas > 0)
);
```

7. Realiza el diseño físico para el siguiente modelo relacional. Asigna el tipo de datos que consideres más adecuado. Realiza el diseño poniendo nombres a las restricciones. Esquema C##TUSINICIALESA7, contraseña "123456".. No pueden ser nulos los siguientes campos: Nombre de Socio, Título de Película. Sexo toma los valores H o M. Por defecto si no se indica nada un actor o actriz no es Protagonista (este campo toma valores S o N). FechaDevolución debe ser mayor que FechaAlquiler.



¿Por qué hay diferentes colores en las tablas?
Los colores ayudan a identificar la naturaleza de la tabla de un vistazo:

- Violeta: Son Entidades Fuertes. Existen por sí solas (Socio, Película, Director). Son las primeras que debes crear.

- Amarillo: Son Tablas de Relación. Representan acciones que unen a dos o más entidades (alquila, actua). Siempre tienen claves foráneas.

- Verde: Son Entidades Débiles. No tienen sentido sin su "padre". Por ejemplo, un Ejemplar no existe si no existe la Película. Su clave primaria suele incluir la de su padre.

Primero, creamos el usuario:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA7 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA7;

ALTER USER C##ASMA7 QUOTA UNLIMITED ON USERS;
```

Segundo, ordenamos las tablas:

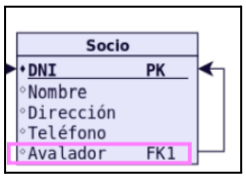
1. Director, Socio y Actores (Las "azules" independientes).
2. Película (Necesita al Director).
3. Ejemplar (Necesita la Película).
4. actua (Relación entre Actor y Película).
5. alquila (Relación entre Socio y Ejemplar).

★ **TABLA DIRECTOR Y SOCIO**

```
--TABLA DIRECTOR Y SOCIO
CREATE TABLE Director (
    Nombre VARCHAR2(50),
    Nacionalidad VARCHAR2(30),
    CONSTRAINT director_pk PRIMARY KEY (Nombre)
);

CREATE TABLE Socio (
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    Avalador VARCHAR2(9),
    CONSTRAINT socio_pk PRIMARY KEY (DNI),
    CONSTRAINT socio_aval_fk FOREIGN KEY (Avalador) REFERENCES Socio (DNI)
);
```

→ Creamos los directores y los socios. Fíjate en el Avalador: es una flecha que vuelve a la misma tabla. Significa que el avalador de un socio debe ser otro socio que ya esté registrado.



★ **TABLA PELÍCULA Y EJEMPLAR**

```
--TABLA PELICULA Y EJEMPLAR
CREATE TABLE Pelicula (
    ID NUMBER(6),
    Titulo VARCHAR2(100) NOT NULL,
    Productora VARCHAR2(50),
    Nacionalidad VARCHAR2(30),
    Fecha DATE,
    Director VARCHAR2(50),
    CONSTRAINT pelicula_pk PRIMARY KEY (ID),
    CONSTRAINT pel_dir_fk FOREIGN KEY (Director) REFERENCES Director (Nombre)
);

CREATE TABLE Ejemplar (
    ID_Peli NUMBER(6),
    Numero NUMBER(3),
    Estado VARCHAR2(20),
    CONSTRAINT ejemplar_pk PRIMARY KEY (ID_Peli, Numero),
    CONSTRAINT eje_pel_fk FOREIGN KEY (ID_Peli) REFERENCES Pelicula (ID)
);
```

→ La película necesita un director real. El Ejemplar: su identidad es el ID de la peli más su número de copia (Copia 1 de la peli 500).

★ **TABLA ACTORES Y ACTÚA**

```
--TABLA ACTORES Y ACTUA

CREATE TABLE Actores (
    Nombre VARCHAR2(50),
    Nacionalidad VARCHAR2(30),
    Sexo CHAR(1),
    CONSTRAINT actores_pk PRIMARY KEY (Nombre),
    CONSTRAINT act_sexo_ck CHECK (Sexo IN ('H', 'M'))
);

CREATE TABLE actua (
    Actor VARCHAR2(50),
    ID_Peli NUMBER(6),
    Prota CHAR(1) DEFAULT 'N',
    CONSTRAINT actua_pk PRIMARY KEY (Actor, ID_Peli),
    CONSTRAINT act_act_fk FOREIGN KEY (Actor) REFERENCES Actores (Nombre),
    CONSTRAINT act_pel_fk FOREIGN KEY (ID_Peli) REFERENCES Pelicula (ID),
    CONSTRAINT act_prota_ck CHECK (Prota IN ('S', 'N'))
);
```

→ En la tabla actua, si no decimos nada, el sistema marca que NO es protagonista (DEFAULT 'N'). Con el CHECK, nos aseguramos de que solo se pueda escribir 'S' o 'N'

-- 1. Creamos la columna "Prota" de 1 solo carácter.
-- 2. "DEFAULT 'N'": Si no insertas nada, Oracle pone 'N' por ti. CONSTRAINT act_prota_ck CHECK (Prota IN ('S', 'N'))
-- 3. "CONSTRAINT act_prota_ck": Le damos un nombre a la regla (policía).
-- 4. "CHECK (... IN ('S', 'N'))": La regla dice que el valor DEBE estar -- dentro de esa lista de dos opciones.

★ **Tabla alquila (La más compleja)**

```
--TABLA ALQUILA
CREATE TABLE alquila (
    DNI VARCHAR2(9),
    ID_Peli NUMBER(6),
    Numero NUMBER(3),
    FechaAlquiler DATE,
    FechaDevolucion DATE,
    CONSTRAINT alquila_pk PRIMARY KEY (DNI, ID_Peli, Numero, FechaAlquiler),
    CONSTRAINT alq_soc_fk FOREIGN KEY (DNI) REFERENCES Socio (DNI),
    CONSTRAINT alq_eje_fk FOREIGN KEY (ID_Peli, Numero) REFERENCES Ejemplar (ID_Peli, Numero),
    CONSTRAINT alq_fechas_ck CHECK (FechaDevolucion > FechaAlquiler)
);
```

→ Esta tabla une al socio con el ejemplar exacto. La regla final es lógica pura: no puedes devolver una película antes de haberla alquilado, por eso el CHECK de fechas.

FECHAS:

Siempre que veas dos columnas de tipo DATE en la misma tabla que representen un proceso (Inicio/Fin, Alquiler/Devolución, Nacimiento/Fallecimiento), hay un CHECK escondido. ¿Puede el evento B ocurrir antes que el evento A?
Si la respuesta es NO: Tienes que poner un CHECK.

Ejemplo del esquema: En la tabla alquila, ves FechaAlquiler y FechaDevolucion. Es físicamente imposible devolver algo antes de alquilarlo.

Sintaxis: `CONSTRAINT nombre_ck CHECK (FechaFinal > FechaInicial)`

NÚMEROS:

Busca columnas que guarden cantidades, precios, edades o stock. ¿Tendría sentido que este número fuera negativo o cero?
Si la respuesta es NO: Necesitas un CHECK.

Ejemplo del esquema: En la tabla NumRevista, el campo NumPaginas. ¿Existe una revista de -5 páginas? No.

Sintaxis: `CONSTRAINT nombre_ck CHECK (NumPaginas > 0)`

TEXTO:

Busca columnas que clasifiquen cosas (Sexo, Categoría, Estado, Turno). ¿Este campo admite cualquier texto o solo unas pocas opciones?

Si el enunciado dice: "Solo puede ser H o M" o "Los valores son Semanal, Mensual...", entonces es un CHECK de lista.

Sintaxis: `CONSTRAINT nombre_ck CHECK (columna IN ('A', 'B', 'C'))`

A veces el esquema no lo dice, pero el texto del enunciado sí.

- "Asegúrese de que el salario sea siempre positivo". → CHECK
- "La fecha de entrega debe ser posterior a la de pedido". → CHECK
- "Solo se admiten socios mayores de 18 años". → CHECK (Edad >= 18)

Código:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA7 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA7;

ALTER USER C##ASMA7 QUOTA UNLIMITED ON USERS;


--TABLA DIRECTOR Y SOCIO
CREATE TABLE Director (
    Nombre VARCHAR2(50),
    Nacionalidad VARCHAR2(30),
    CONSTRAINT director_pk PRIMARY KEY (Nombre)
);

CREATE TABLE Socio (
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    Direccion VARCHAR2(100),
    Telefono NUMBER(9),
    Avalador VARCHAR2(9),
    CONSTRAINT socio_pk PRIMARY KEY (DNI),
    CONSTRAINT socio_aval_fk FOREIGN KEY (Avalador) REFERENCES
Socio (DNI)
);

--TABLA PELÍCULA Y EJEMPLAR
CREATE TABLE Pelicula (
    ID NUMBER(6),
    Titulo VARCHAR2(100) NOT NULL,
    Productora VARCHAR2(50),
    Nacionalidad VARCHAR2(30),
    Fecha DATE,
    Director VARCHAR2(50),
    CONSTRAINT pelicula_pk PRIMARY KEY (ID),
    CONSTRAINT pel_dir_fk FOREIGN KEY (Director) REFERENCES
Director (Nombre)
);

CREATE TABLE Ejemplar (
    ID_Peli NUMBER(6),
    Numero NUMBER(3),
    Estado VARCHAR2(20),
    CONSTRAINT ejemplar_pk PRIMARY KEY (ID_Peli, Numero),
    CONSTRAINT eje_pel_fk FOREIGN KEY (ID_Peli) REFERENCES
Pelicula (ID)
```

```
);

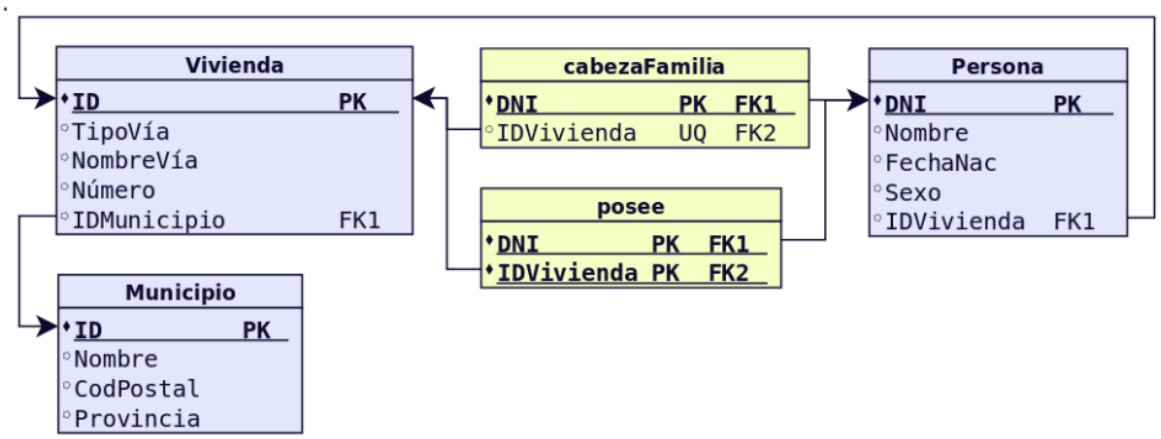

--TABLA ACTORES Y ACTUA

CREATE TABLE Actores (
    Nombre VARCHAR2(50),
    Nacionalidad VARCHAR2(30),
    Sexo CHAR(1),
    CONSTRAINT actores_pk PRIMARY KEY (Nombre),
    CONSTRAINT act_sexo_ck CHECK (Sexo IN ('H', 'M'))
);

CREATE TABLE actua (
    Actor VARCHAR2(50),
    ID_Peli NUMBER(6),
    Prota CHAR(1) DEFAULT 'N',
    CONSTRAINT actua_pk PRIMARY KEY (Actor, ID_Peli),
    CONSTRAINT act_act_fk FOREIGN KEY (Actor) REFERENCES Actores
(Nombre),
    CONSTRAINT act_pel_fk FOREIGN KEY (ID_Peli) REFERENCES
Pelicula (ID),
    CONSTRAINT act_prota_ck CHECK (Prota IN ('S', 'N'))
);


--TABLA ALQUILA
CREATE TABLE alquila (
    DNI VARCHAR2(9),
    ID_Peli NUMBER(6),
    Numero NUMBER(3),
    FechaAlquiler DATE,
    FechaDevolucion DATE,
    CONSTRAINT alquila_pk PRIMARY KEY (DNI, ID_Peli, Numero,
FechaAlquiler),
    CONSTRAINT alq_soc_fk FOREIGN KEY (DNI) REFERENCES Socio
(DNI),
    CONSTRAINT alq_eje_fk FOREIGN KEY (ID_Peli, Numero) REFERENCES
Ejemplar (ID_Peli, Numero),
    CONSTRAINT alq_fechas_ck CHECK (FechaDevolucion >
FechaAlquiler)
);
```

8. Realiza el diseño físico para el siguiente modelo relacional. Asigna el tipo de datos que consideres más adecuado. Realiza el diseño poniendo nombres a las restricciones. Esquema C##TUSINICIALESA8, contraseña "123456". No pueden ser nulos los siguientes campos: Nombre de Persona, NombreVía, Número de Vivienda, Nombre de Municipio. Sexo toma los valores H o M. Por defecto si no se indica nada el TipoVia es Calle



Primero, creamos el usuario.

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA8 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA8;

ALTER USER C##ASMA8 QUOTA UNLIMITED ON USERS;
```

Segundo, ordenamos las tablas:

1. **Municipio**: No depende de nadie.
2. **Vivienda**: Depende de Municipio.
3. **Persona**: Depende de Vivienda (donde vive).
4. **posee**: Une Persona y Vivienda (propiedad).
5. **cabezaFamilia**: La relación especial 1 a 1.

★ **TABLA MUNICIPIO**

```
--TABLA MUNICIPIO
CREATE TABLE Municipio (
    ID NUMBER(5),
    Nombre VARCHAR2(50) NOT NULL,
    CodPostal NUMBER(5),
    Provincia VARCHAR2(30),
    CONSTRAINT municipio_pk PRIMARY KEY (ID)
);
```

→Primero creamos los pueblos. El ID es la llave. El Nombre es obligatorio porque un municipio sin nombre no tiene sentido.

★ **TABLA VIVIENDA**

```
--TABLA VIVIENDA
CREATE TABLE Vivienda (
    ID NUMBER(10),
    TipoVia VARCHAR2(20) DEFAULT 'Calle',
    NombreVia VARCHAR2(50) NOT NULL,
    Numero NUMBER(4) NOT NULL,
    IDMunicipio NUMBER(5),
    CONSTRAINT vivienda_pk PRIMARY KEY (ID),
    CONSTRAINT viv_mun_fk FOREIGN KEY (IDMunicipio) REFERENCES Municipio(ID)
);
```

→ Creamos las casas. Aplicamos el DEFAULT 'Calle': si no dices nada, el sistema asume que es una calle. El nombre de la vía y el número son obligatorios por contrato (NOT NULL).

★ **TABLA PERSONA**

```
--TABLA PERSONA
CREATE TABLE Persona (
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    FechaNac DATE,
    Sexo CHAR(1),
    IDVivienda NUMBER(10),
    CONSTRAINT persona_pk PRIMARY KEY (DNI),
    CONSTRAINT per_viv_fk FOREIGN KEY (IDVivienda) REFERENCES Vivienda(ID),
    CONSTRAINT per_sexo_ck CHECK (Sexo IN ('H', 'M'))
);
```

→ La lista de ciudadanos. Obligatorio el nombre. Ponemos al 'policía' (CHECK) para que el sexo solo sea 'H' o 'M'. Además, anotamos en qué casa vive mediante la FK.

★ **TABLA POSEE**

```
--TABLA POSEE
CREATE TABLE posee (
    DNI VARCHAR2(9),
    IDVivienda NUMBER(10),
    CONSTRAINT posee_pk PRIMARY KEY (DNI, IDVivienda),
    CONSTRAINT pos_per_fk FOREIGN KEY (DNI) REFERENCES Persona(DNI),
    CONSTRAINT pos_viv_fk FOREIGN KEY (IDVivienda) REFERENCES Vivienda(ID)
);
```

→ Es una relación de N:M. Esta tabla es para saber quién es el dueño de cada casa. Como una persona puede tener varias casas y una casa varios dueños, la PK es la pareja de ambos datos.

★ **TABLA CABEZAFAMILIA**

```
--TABLA CABEZAFAMILIA
CREATE TABLE cabezaFamilia (
    DNI VARCHAR2(9),
    IDVivienda NUMBER(10),
    CONSTRAINT cab_pk PRIMARY KEY (DNI),
    CONSTRAINT cab_per_fk FOREIGN KEY (DNI) REFERENCES Persona(DNI),
    CONSTRAINT cab_viv_fk FOREIGN KEY (IDVivienda) REFERENCES Vivienda (ID),
    CONSTRAINT cab_viv_uq UNIQUE (IDVivienda)
);
```

→ Es una relación 1:1. Esta es especial. El DNI es la PK (solo hay un registro por cabeza de familia). Pero ojo al UNIQUE (IDVivienda): esto obliga a que cada casa solo pueda tener un único cabeza de familia registrado. ¡No vale repetir casa!

Código:

```
-- CREACIÓN DEL USUARIO
--
CREATE USER C##ASMA8 IDENTIFIED BY "123456";

GRANT CONNECT, RESOURCE TO C##ASMA8;

ALTER USER C##ASMA8 QUOTA UNLIMITED ON USERS;

--TABLA MUNICIPIO
CREATE TABLE Municipio (
    ID NUMBER(5),
```

```
    Nombre VARCHAR2(50) NOT NULL,
    CodPostal NUMBER(5),
    Provincia VARCHAR2(30),
    CONSTRAINT municipio_pk PRIMARY KEY (ID)
);


--TABLA VIVIENDA
CREATE TABLE Vivienda (
    ID NUMBER(10),
    TipoVia VARCHAR2(20) DEFAULT 'Calle',
    NombreVia VARCHAR2(50) NOT NULL,
    Numero NUMBER(4) NOT NULL,
    IDMunicipio NUMBER(5),
    CONSTRAINT vivienda_pk PRIMARY KEY (ID),
    CONSTRAINT viv_mun_fk FOREIGN KEY (IDMunicipio) REFERENCES
Municipio(ID)
);



--TABLA PERSONA
CREATE TABLE Persona (
    DNI VARCHAR2(9),
    Nombre VARCHAR2(50) NOT NULL,
    FechaNac DATE,
    Sexo CHAR(1),
    IDVivienda NUMBER(10),
    CONSTRAINT persona_pk PRIMARY KEY (DNI),
    CONSTRAINT per_viv_fk FOREIGN KEY (IDVivienda) REFERENCES
Vivienda(ID),
    CONSTRAINT per_sexo_ck CHECK (Sexo IN ('H', 'M'))
);

--TABLA POSEE
CREATE TABLE posee (
    DNI VARCHAR2(9),
    IDVivienda NUMBER(10),
    CONSTRAINT posee_pk PRIMARY KEY (DNI, IDVivienda),
    CONSTRAINT pos_per_fk FOREIGN KEY (DNI) REFERENCES
Persona(DNI),
    CONSTRAINT pos_viv_fk FOREIGN KEY (IDVivienda) REFERENCES
Vivienda(ID)
);
```

```
--TABLA CABEZAFAMILIA
CREATE TABLE cabezaFamilia (
    DNI VARCHAR2(9),
    IDVivienda NUMBER(10),
    CONSTRAINT cab_pk PRIMARY KEY (DNI),
    CONSTRAINT cab_per_fk FOREIGN KEY (DNI) REFERENCES
Persona(DNI),
    CONSTRAINT cab_viv_fk FOREIGN KEY (IDVivienda) REFERENCES
Vivienda (ID),
    CONSTRAINT cab_viv_uq UNIQUE (IDVivienda)
);
```